KLASIFIKASI CEPAT MODEL *XCEPTIONNET* DAN *RESNET*-50 PADA VIDEO *DEEPFAKE* MENGGUNAKAN *LOCAL BINARY PATTERN*

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh Gelar

Sarjana Komputer (S.Kom.)

Oleh :

Javid Al Haq

NIM : 11160910000002

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI SYARIF HIDAYATULLAH JAKARTA

2021 M / 1442 H

KLASIFIKASI CEPAT MODEL *XCEPTIONNET* DAN *RESNET*-50 PADA VIDEO *DEEPFAKE* MENGGUNAKAN *LOCAL BINARY PATTERN*

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh Gelar

Sarjana Komputer (S.Kom)

Oleh :

Javid Al Haq

NIM : 11160910000002

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI SYARIF HIDAYATULLAH JAKARTA

2021 M / 1442 H

LEMBAR PERSETUJUAN

KLASIFIKASI CEPAT
MODEL *XCEPTIONNET* DAN *RESNET*-50 PADA VIDEO *DEEPFAKE*
MENGGUNAKAN *LOCAL BINARY PATTERN*

Skripsi

Sebagai Salah Satu Syarat untuk Memperoleh

Gelar Sarjana Komputer (S.Kom)

Oleh :

Javid Al Haq
11160910000002

Menyetujui,

Pembimbing I	**Pembimbing II**
Arini, S.T., M.T.	**Rizal M.Kom.**
NIP : 197601312009012001	NIP: 197108062014111001

Mengetahui,

Ketua Prodi Teknik Informatika

Dr. Imam Marzuki Shofi, M.T.

NIP : 197202052008011010

PENGESAHAN UJIAN

Skripsi yang berjudul "Klasifikasi Cepat Model *Xceptionnet* Dan *Resnet*-50 Pada Video *Deepfake* Menggunakan *Local Binary Pattern*" yang ditulis oleh Javid Al Haq, NIM 11160910000002 telah diuji dan dinyatakan lulus dalam sidang munaqasyah Fakultas Sains dan Teknologi UIN Syarif Hidayatullah Jakarta pada 17 November 2021. Skripsi ini telah diterima sebagai salah satu syarat memperoleh gelar Sarjana Komputer (S.Kom) pada program Studi Teknik Informatika.

Jakarta, Desember 2021

Tim Penguji

Penguji I	**Penguji II**
Hendra Bayu Suseno, M.Kom.	**Fitri Mintarsih, M.Kom.**
NIP : 198212112009121003	NIP: 197212232007102004

Tim Pembimbing,

Pembimbing I	**Pembimbing II**
Arini, S.T., M.T.	**Rizal M.Kom.**
NIP : 197601312009012001	NIP: 197108062014111001

Mengetahui,

Dekan FST	**Ketua Program Studi**
Dr. Nashrul Hakiem, S.Si., M.T., Ph.D.	**Dr. Imam Marzuki Shofi, MT.**
NIP : 197106082005011005	NIP : 197202052008011010

PERNYATAAN ORISINALITAS

Dengan ini saya menyatakan bahwa :

1. Skripsi ini merupakan hasil karya asli saya yang diajukan untuk memenuhi salah satu persyaratan memperoleh gelar Strata 1 di UIN Syarif Hidayatullah Jakarta.
2. Semua sumber yang saya gunakan dalam penulisan ini telah saya cantumkan sesuai dengan ketentuan yang berlaku di UIN Syarif Hidayatullah Jakarta.
3. Apabila di kemudian hari terbukti karya ini bukan karya asli saya atau merupakan hasil jiplakan karya orang lain, maka saya bersedia menerima sanksi yang berlaku di UIN Syarif Hidayatullah Jakarta.

Jakarta, Desember 2021

Penulis

Javid Al Haq

NIM : 11160910000002

UIN Syarif Hidayatullah Jakarta

PERNYATAAN PERSETUJUAN PUBLIKASI SKRIPSI

Sebagai civitas akademik UIN Syarif Hidayatullah Jakarta, saya yang bertanda tangan dibawah ini :

Nama	: Javid Al Haq
NIM	: 11160910000002
Program Studi	: Teknik Informatika
Fakultas	: Sains dan Teknologi
Jenis Karya	: Skripsi

Demi pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Islam Negeri Syarif Hidayatullah Jakarta **Hak Bebas Royalti Noneksklusif (*Non-exclusive Royalty Free Right*)** atas karya ilmiah saya yang berjudul :

KLASIFIKASI CEPAT
MODEL *XCEPTIONNET* DAN *RESNET*-50 PADA VIDEO *DEEPFAKE*
MENGGUNAKAN *LOCAL BINARY PATTERN*

Beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Islam Negeri Syarif Hidayatullah Jakarta berhak menyimpan, mengalih media/formatkan, mengelola dalam bentuk pangkalan data (database), merawat dan mempublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Jakarta

Pada tanggal : 20 Desember 2021

Yang menyatakan

(Javid Al Haq)

Nama : **Javid Al Haq**

Program Studi : **Teknik Informatika**

Judul : **Klasifikasi Cepat Model *XceptionNet* Dan *ResNet*-50 Pada Video *Deepfake* Menggunakan *Local Binary Pattern***

ABSTRAK

Kemudahan dalam pembuatan video *deepfake* menunjukkan bahwa kebutuhan metode otentikasi yang semakin meningkat untuk menklasifikasi video *deepfake*. Masalah ini sangat sensitif, *deepfake* memungkinkan untuk membuat seseorang mengatakan atau melakukan sesuatu yang tidak pernah mereka katakan atau lakukan dalam sebuah video, hal ini membahayakan martabat seseorang. Masalah lain muncul dengan besarnya ukuran dataset video *deepfake*, karena dengan ukuran dataset yang besar membutuhkan waktu pelatihan yang lebih lama dan spesifikasi komputasi yang tinggi . Dalam penelitian ini, penulis mengusulkan metode yang menggunakan dataset berjumlah total 2000 *frame* dari keseluruhan dataset Celeb-DF(V2) dengan menerapkan (*Multi-task Cascaded Convolutional Networks*) MTCNN untuk pengambilan data wajah pada *frame* video, pemrosesan gambar *filter Gaussian* dan *Local Binary Pattern* (LBP) dengan model *XceptionNet* dan *ResNet*-50 sebagai pembanding untuk klasifikasi video *deepfake*. Hasil menunjukkan bahwa metode yang diusulkan memiliki performa yang lebih baik dibandingkan dengan pemrosesan gambar filter *Gaussian* dan LBP dengan model *ResNet*-50, dengan nilai AUC sebesar 0.87 dan nilai akurasi sebesar 0.79.

Kata Kunci : Klasifikasi video, *Deepfake*, *XceptionNet*, *ResNet*-50, Dataset, *Artificial Intelligence*, MTCNN, *Filter Gaussian*, *Local Binary Pattern*

Daftar Pustaka : 76 (Tahun 2014 - 2021)

Jumlah Halaman : VI BAB + XV Halaman + 71 Halaman + 1 Lampiran

Nama : **Javid Al Haq**
Program Studi : **Teknik Informatika**
Judul : **Klasifikasi Cepat Model *XceptionNet* Dan *ResNet*-50 Pada Video *Deepfake* Menggunakan *Local Binary Pattern***

ABSTRACT

The ease of creating deepfake videos indicates that there is an increasing need for authentication methods to classify deepfake videos. This issue is very sensitive, deepfakes make it possible to make someone say or do something they never said or did in a video, this puts a person's dignity at risk. Another problem arises with the large size of the deepfake video dataset, due to the size of the dataset which requires longer training times and high computational specifications In this study, the authors propose a method that uses a dataset of a total of 2000 frames from the entire Celeb-DF(V2) dataset by applying (Multi-task Cascaded Convolutional Networks) MTCNN for facial data retrieval on video frames, image processing with Gaussian filters and Local Binary Patterns. (LBP) with XceptionNet and ResNet-50 models as a comparison for deepfake video classification. The results show that the proposed method has a better performance than the Gaussian filter and LBP image processing with the ResNet-50 model, with an AUC value of 0.87 and an accuracy value of 0.79.

Keywords : *Video Classification*, *Deepfake*, *XceptionNet*, *ResNet*-50, *Dataset*, *Artificial Intelligence*, MTCNN, *Filter Gaussian*, *Local Binary Pattern*
Daftar Pustaka : 76 (Year 2014 - 2021)
Jumlah Halaman : VI BAB + XV Pages + 71 Pages + 1 Attachment

KATA PENGANTAR

Assalamualaikum Wr.Wb

Segala puji dan syukur penulis panjatkan kehadirat Allah SWT atas limpahan rahmat, nikmat, taufiq dan karunia serta pertolongannya-Nya,sehingga skripsi ini dapat terselesaikan pada waktu yang tepat. Sholawat serta salam selalu tercurahkan kepada kekasih Allah, suri tauladan dan rahmat bagi seluruh alam, yaitu junjungan kita Nabi Muhammad SAW beserta keluarga, sahabatnya, dan seluruh umatnya hingga akhir zaman. Penyusunan skripsi ini adalah salah satu syarat untuk memperoleh gelar Sarjana Komputer (S.Kom) pada program studi Teknik Informatika, Fakultas Sains dan Teknologi, Universitas Islam Negeri Syarif Hidayatullah Jakarta. skripsi ini kedepannya diharapkan mampu memberikan input (masukan) yang positif, dan adanya perubahan ke arah yang lebih baik. Dalam proses penulisan skripsi ini, penulis mendapatkan bantuan, dukungan, saran dan dari berbagai pihak. Oleh karena itu, penulis ingin mengucapkan terimakasih yang sebesar - sebesarnya kepada :

1. Orang Tua tercinta yang senantiasa memberikan dukungan moril dan materi, juga kakak dan adik tersayang. Tiada tutur kata selain terima kasih kepada kalian. Terima kasih, *Alhamdulillah.*
2. Bapak Ir. Nashrul Hakiem, S.Si., M.T., Ph.D selaku dekan Fakultas Sains dan Teknologi.
3. Bapak Imam Marzuki Shofi, M.T. selaku Ketua Program Studi Teknik Informatika.
4. Ibu Arini, S.T., M.T. sebagai Dosen Pembimbing I yang telah membimbing selama penulisan skripsi ini
5. Bapak Rizal Broer Bahaweres, M.Kom. sebagai Dosen Pembimbing II yang telah memberikan masukan dan arahan dalam penulisan skripsi ini
6. Bapak Husni Teja Sukmana, Ph.D sebagai Dosen Pembimbing Akademik penulis.

7. Seluruh Dosen dan Staff Karyawan Fakultas Sains dan Teknologi UIN Syarif Hidayatullah Jakarta yang telah memberikan banyak ilmu dan dukungan selama masa perkuliahan.
8. Teman-teman Teknik Informatika TI 2016.
9. Serta seluruh pihak yang telah membantu penulis secara langsung maupun tidak langsung yang tidak dapat disebutkan namanya satu persatu.

Penulisan skripsi ini masih jauh dari kata sempurna. Untuk itu, sangat diperlukan kritik dan saran yang membangun bagi penulis. Akhir kata, semoga laporan skripsi ini dapat bermanfaat bagi penulis dan orang lain.

Wassalamualaikum, Wr. Wb.

Jakarta, November 2021

Penulis

Javid Al Haq

11160910000002

DAFTAR ISI

LEMBAR PERSETUJUAN i
PENGESAHAN UJIAN ii
PERNYATAAN ORISINALITAS iii
PERNYATAAN PERSETUJUAN PUBLIKASI SKRIPSI iv
ABSTRAK v
ABSTRACT vi
KATA PENGANTAR vii
DAFTAR ISI ix
DAFTAR TABEL xii
DAFTAR GAMBAR xiii
DAFTAR LAMPIRAN xv
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 4
1.3 Batasan Masalah 4
1.3.1 Proses 4
1.3.2 Metode 5
1.3.3 Tools 5
1.4 Tujuan Penelitian 6
1.5 Manfaat Penulisan 6
1.6 Metodologi penelitian 6
1.6.1 Metode Pengumpulan Data 7
1.7 Sistematika Penulisan 7
BAB II LANDASAN TEORI 9
2.1 Klasifikasi Gambar 9
2.2 Deepfake 9
2.3 Dataset 10
2.3.1 Celeb-DF(V2) 11
2.4 Pengolahan Citra Digital 12

2.4.1 Image Enhancement 12
2.4.2 Local Binary Pattern 13
2.4.3 Filter Gaussian 14
2.5 Kecerdasan Buatan 14
2.5.1 Machine Learning 15
2.5.2 Deep Learning 16
2.6 Tensorflow 17
2.7 Convolutional Neural Network 17
2.7.1 Convolutional Layer 18
2.7.2 Pooling Layer 18
2.7.3 Fully Connected Layer 19
2.8 MTCNN 20
2.9 Keras 22
2.9.1 *XceptionNet* 22
2.9.2 *ResNet*-50 23
2.10 Area Dibawah Kurva (AUC) 24
2.11 Confusion Matrix 26
2.12 F1 Score 27
2.13 Google Colaboratory 27
2.14 Data Science Methodology 28
2.15 Penelitian Terkait 31
BAB III METODE PENELITIAN 36
3.1 Kerangka Berpikir 36
3.2 Metode Pengumpulan Data 37
3.2.1 Studi Pustaka 37
3.2.2 Observasi 37
3.3 Metode Implementasi Eksperimen 37
BAB IV IMPLEMENTASI 40
4.1 Business Understanding dan Pendekatan Analisis 40
4.2 Analytic Approach 40
4.3 Data Requirement 40

4.4 Data Collection 41
4.4.1 Ekstraksi Wajah 42
4.4.2 Proses Filter Gaussian dan Local Binary Pattern 42
4.4.3 Dataset 43
4.5 Data Understanding 43
4.6 Data Preparation 43
4.7 Modeling 44
4.7.1 Pelatihan (*Training*) 46
BAB V HASIL DAN PEMBAHASAN 51
5.1 Evaluasi 51
5.1.1 Hasil Pelatihan Model *XceptionNet* 51
5.1.2 Hasil Pelatihan Model *ResNet*-50 53
5.1.3 Bagaimana performa metode *Filter Gaussian* dan LBP dengan model *XceptionNet* dan model *ResNet*-50 sebagai pembanding dalam klasifikasi video *deepfake*? 54
5.1.4 Pengujian 57
BAB VI PENUTUP 61
6.1 Kesimpulan 61
6.2 Saran 61
DAFTAR PUSTAKA 62
LAMPIRAN 72

UIN Syarif Hidayatullah Jakarta

DAFTAR TABEL

Tabel 2.1 : Literatur Sejenis 33
Tabel 4.1 : Tabel Keterangan Sumber Dataset 41
Tabel 4.2 : Keterangan Dataset 43
Tabel 4.3 : Tabel Pembagian Dataset 44
Tabel 5.1 : Tabel Hasil Pelatihan Model XceptionNet 51
Tabel 5.2 : Tabel Hasil Pelatihan Model ResNet-50 53
Tabel 5.3 : Tabel Perbandingan Performa Model Pada Set Validasi 55
Tabel 5.4 : Tabel Perbandingan Confusion Matrix 56
Tabel 5.5 : Tabel Perbandingan Keseluruhan Performa Model 56
Tabel 5.6 : Tabel Hasil Klasifikasi Model XceptionNet Pada Video Real 58
Tabel 5.7 : Tabel Hasil Klasifikasi Model XceptionNet Pada Video Deepfake ... 58
Tabel 5.8 : Tabel Hasil Klasifikasi Model ResNet-50 Pada Video Real 59
Tabel 5.9 : Tabel Hasil Klasifikasi Model ResNet-50 Pada Video Deepfake 59
Tabel 5.10 : Tabel Perbandingan Hasil Pengujian Klasifikasi Model 60

DAFTAR GAMBAR

Gambar 2.1 : Contoh Dataset Celeb-DF(V2) dengan Kotak Hijau merupakan wajah asli dan kotak merah merupakan wajah deepfake 11
Gambar 2.2 : Informasi Dasar Dataset Celeb-DF(V2)..... 11
Gambar 2.3 : Tahapan Perhitungan LBP 13
Gambar 2.4 : Diagram Deep Learning..... 16
Gambar 2.5 : Lapisan Konvolusi ..... n 18
Gambar 2.6 : Pooling Layer..... 19
Gambar 2.7 : Fully Connected Layer..... 19
Gambar 2.8 : Arsitektur MTCNN..... 20
Gambar 2.9 : Langkah-langkah dari MTCNN 21
Gambar 2.10 : spesifikasi Model XceptionNet 22
Gambar 2.11 : Arsitektur Model XceptionNet..... 23
Gambar 2.12 : Spesifikasi Model ResNet-50..... 23
Gambar 2.13 : Arsitektur Model ResNet-50 24
Gambar 2.14 : Aturan Penafsiran Nilai AUC 25
Gambar 2.15 : Contoh Confusion Matrix 26
Gambar 2.16 : Diagram Data Science Methodology 28
Gambar 3.1 : Kerangka Berpikir..... 36
Gambar 3.2 : Gambar Diagram Data science methodology yang digunakan..... 37
Gambar 4.1 : Contoh ekstraksi wajah dari frame menggunakan MTCNN..... 42
Gambar 4.2 : Contoh Sebelum dan Sesudah Penerapan Proses Filter Gaussian dan LBP 43
Gambar 4.3 : Alur Metode 45
Gambar 4.4 : Alur Skenario Proses Eksperimen..... 46
Gambar 4.5 : Label dan Kelas Data 47
Gambar 5.1 : Grafik Hasil Pelatihan Model XceptionNet dengan warna oranye merupakan hasil pada set validasi dan biru pada set pelatihan. Grafik akurasi model berada pada bagian kiri, dan loss model pada bagian kanan..... 51
Gambar 5.2 Classification Report Model XceptionNet 52
Gambar 5.3 : Confusion Matrix Model XceptionNet 52
Gambar 5.4 : Grafik Hasil Pelatihan Model ResNet-50 dengan warna oranye merupakan hasil pada set validasi dan biru pada set pelatihan. Grafik akurasi model berada pada bagian kiri, dan loss model pada bagian kanan..... 53
Gambar 5.5 : Classification Report Model ResNet-50 53
Gambar 5.6 : Confusion Matrix Model ResNet-50..... 54
Gambar 5.7 : Grafik perbandingan hasil pelatihan pada set validasi. Warna oranye merupakan model ResNet-50 dan biru merupakan model XceptionNet. Grafik

perbandingan akurasi model berada pada bagian kiri, dan perbandingan loss model pada bagian kanan. 54
Gambar 5.8 : Grafik perbandingan performa nilai AUC model. Warna oranye merupakan model ResNet-50 dan biru merupakan model XceptionNet. 55
Gambar 5.9 : Perbandingan performa model pada Confusion Matrix. Confusion matrix model XceptionNet berada pada bagian kiri, dan Confusion matrix model ResNet-50 pada bagian kanan. 55
Gambar 5.10 : Grafik Perbandingan Keseluruhan Performa Model 56
Gambar 5.11 : Gambar Hasil Klasifikasi Real Pada Frame 57
Gambar 5.12 : Gambar Hasil Klasifikasi Deepfake Pada Frame 57

UIN Syarif Hidayatullah Jakarta

DAFTAR LAMPIRAN

Lampiran 1 : Lampiran Source Code Fungsi Klasifikasi Video Deepfake..... 72

UIN Syarif Hidayatullah Jakarta

BAB I
PENDAHULUAN

1.1 Latar Belakang

Teknologi terus mengalami kemajuan seiring dengan perkembangan zaman. Hal tersebut memberikan dampak positif untuk kehidupan manusia, namun di sisi lain berdampak negatif karena ada pihak yang menyalah gunakan perkembangan teknologi tersebut dengan merugikan pihak lain. Termasuk dari perkembangan teknologi tersebut adalah dalam bidang multimedia. Teknik manipulasi dan evolusi teknologi multimedia memungkinkan bagi seseorang bahkan seorang pemula untuk dengan mudah menghapus objek ataupun menambahkan objek ke dalam video (Al-Sanjary & Sulong, 2015).

Deepfake berasal dari "*deep learning*" dan "*fake*" adalah teknik yang dapat menempatkan gambar wajah seorang target, ke video orang lain untuk membuat video orang yang menjadi target melakukan atau mengatakan hal-hal yang dilakukan orang lain (Nguyen et al., 2019). Maras & Alexandrou (2019) menyatakan video *deepfake* merupakan produk kecerdasan buatan atau aplikasi pembelajaran mesin yang menggabungkan, mengganti, dan melapiskan gambar dan klip video ke dalam video, membuat video palsu menjadi tampak asli. Teknik *deepfake* memungkinkan untuk membuat sebuah video tampak seolah-olah seseorang mengatakan atau melakukan sesuatu yang tidak pernah mereka katakan atau lakukan. *Deepfake* dapat digunakan untuk hal yang membahayakan, termasuk serangan terhadap martabat seseorang dan menciptakan ketidakstabilan politik (Feeney, 2021).

Berdasarkan laporan Henry Ajder & Giorgio Patrini (2019) pada tahun 2019 jumlah total video *deepfake online* adalah 14.678, menunjukkan bahwa jumlah total video *deepfake online* meningkat pesat, dengan peningkatan hampir 100% dibandingkan pengukuran sebelumnya yaitu 7.964 yang diambil pada bulan Desember 2018. Kemudahan dalam pembuatan video *deepfake* menimbulkan tantangan forensik yang unik. Menunjukkan bahwa kebutuhan metode otentikasi

yang semakin meningkat untuk mendeteksi *deepfake*. Masalah ini sangat sensitif dan mendesak di era 'berita palsu' saat ini, terkait penegakan hukum dan juga bagi wartawan, situs web hosting video, dan pengguna media sosial (Rodriguez et al., 2018). Pendeteksian *deepfake* tidak mudah untuk dilakukan manusia, Almars (2021) menyatakan *deepfake* menggunakan teknologi *deep learning* untuk memanipulasi gambar, dan video *deepfake* tersebut keasliannya tidak dapat dengan mudah dibedakan oleh mata manusia.

Besarnya ukuran dataset video *deepfake* menjadi masalah lain, karena dengan ukuran dataset yang besar membutuhkan waktu pelatihan yang lama dan spesifikasi komputasi yang tinggi. Althnian (2021) menyatakan dalam kasus kumpulan data besar, melatih model membutuhkan waktu dan sumber daya komputasi lebih lanjut, yang mungkin tidak tersedia. Menurut Francois Chollet (2016) *deep learning* membutuhkan kemampuan untuk mempelajari fitur secara otomatis dari data, yang umumnya hanya mungkin jika banyak data pelatihan tersedia, terutama untuk masalah di mana sampel masukan berdimensi sangat tinggi, seperti gambar. Namun, *convolutional neural networks* (CNN) yang merupakan algoritma pilar *deep learning* dirancang sebagai salah satu model terbaik yang tersedia untuk sebagian besar masalah "persepsi" (seperti klasifikasi gambar), bahkan dengan sangat sedikit data untuk dipelajari. Melatih CNN dari awal pada kumpulan data gambar kecil akan tetap memberikan hasil yang wajar, tanpa memerlukan rekayasa fitur khusus apa pun.

Menurut Cordeiro de Amorim (2019) pada dekade terakhir ketersediaan data memiliki peningkatan yang cukup besar. Sayangnya, peningkatan ini dibayangi oleh berbagai kesulitan teknis yang muncul saat menganalisis kumpulan data yang besar. Ini termasuk waktu pemrosesan yang lama, persyaratan besar untuk penyimpanan data, dan masalah teknis lainnya yang terkait dengan analisis kumpulan data dimensi tinggi. Dalam banyak kasus penggunaan di dunia nyata, bahkan pengumpulan data skala kecil bisa sangat mahal atau terkadang hampir tidak mungkin (misalnya dalam pencitraan medis). Mampu memanfaatkan data

yang sangat sedikit adalah keterampilan utama seorang ilmuwan data yang kompeten (Francois Chollet, 2016).

Menurut Almars (2021) *Deep learning* telah menunjukkan hasil yang menjanjikan dalam mendeteksi *deepfake*, dalam penelitian sebelumnya, beberapa teknik menggunakan *deep learning* telah diusulkan, termasuk : CNN, *recurrent neural network* (RNN), *long short-term memory* (LSTM). Pada beberapa peneltian sebelumnya tentang klasifikasi video *deepfake*, *deep learning* banyak digunakan. Penelitian oleh Rössler (2019) menunjukkan hasil model *XceptionNet* Chollet (2017) memilki performa akurasi 96.36%, yaitu lebih tinggi dibandingkan dengan penelitian sebelumnya seperti MesoNet Afchar (2019), model yang diusulkan Rahmouni (2017), model yang diusulkan Bayar & Stamm (2016), dan model yang diusulkan Cozzolino (2017). Selain itu penggunaan *deep learning* dalam mendeteksi *deepfake* terdapat pada penelitian Li & Lyu (2018) yang menggunakan model *ResNet*-50, VGG16, *ResNet*101 and *ResNet*152 dengan hasil *ResNet*-50 sebagai model terbaik.

Kaur & Kaur (2015) menyatakan pengolahan citra adalah proses perbaikan citra atau fitur-fiturnya untuk mendapatkan detail yang maksimal dan menonjolkan informasi yang bermanfaat. Menurut H. Zhang (2017) LBP (*Local Binary Pattern*) adalah operator yang digunakan untuk menggambarkan fitur tekstur lokal dari gambar, LBP memiliki keunggulan pada invarian rotasi dan invarian abu-abu. Penggunaan LBP untuk pengenalan, klasifikasi objek, dan klasifikasi gambar sudah banyak digunakan. LBP adalah deskriptor yang sederhana namun kuat untuk fitur spasial, dan dapat mengurangi beban kerja CNN serta meningkatkan akurasi klasifikasi (Wei et al., 2019). Selain menggunakan LBP, *filter Gaussian* juga digunakan dalam penelitian ini. *Filter Gaussian* sudah umum digunakan dalam pemrosesan gambar. Menurut Padmaja (2020) keuntungan utama menggunakan filter *Gaussian* adalah lebih cepat karena menduplikasi dan memasukkan estimasi piksel gambar lebih cepat daripada menyortir dan menyusunnya.

Penggunaan *Multi-task Cascaded Convolutional Networks* (MTCNN) yang diusulkan oleh Zhang (2016) untuk masalah pendeteksian wajah sudah banyak digunakan. Menurut Du (2020) MTCNN mengusulkan kerangka kerja ringan baru berbasis CNN berjenjang untuk deteksi dan penyelarasan wajah. Berdasarkan hasil percobaan, produksi MTCNN sangat bagus untuk hasil deteksi wajah *real time* dengan presisi tinggi.

Berdasarkan permasalahan diatas penulis mengusulkan metode yang menggunakan dataset dengan jumlah total 2000 *frame* dari keseluruhan dataset Celeb-DF dengan menerapkan pemrosesan gambar *filter Gaussian* dan LBP dengan model *XceptionNet*, dan model *ResNet*-50 sebagai pembanding untuk klasifikasi video *deepfake*. Penulis memilih model *ResNet*-50 sebagai dasar perbandingan pengaruh performa model yang menggunakan pemrosesan gambar *filter Gaussian* dan LBP. peneliti akan melakukan penelitian yang berjudul **"Klasifikasi Cepat Model *XceptionNet* Dan *ResNet*-50 Pada Video *Deepfake* Menggunakan *Local Binary Pattern* "**.

1.2 Rumusan Masalah

"Bagaimana performa metode Filter *Gaussian* dan LBP dengan model *XceptionNet* dan model *ResNet*-50 sebagai pembanding dalam klasifikasi video *deepfake* dengan dataset yang berjumlah 2000 *frame* dari keseluruhan dataset Celeb-DF(V2)?"

1.3 Batasan Masalah

Dalam penelitian ini, peneliti melakukan pembatasan masalah terhadap masalah penelitian yang akan dilakukan, yakni :

1.3.1 Proses

Berikut ini adalah proses yang terdapat pada skripsi,yaitu :

1. Penelitian ini dibuat dengan ruang lingkup video tanpa suara, dengan format ".mp4" dan gambar dengan format ".jpg".
2. Dataset dasar menggunakan dataset Celeb-DF(V2).

3. Karena keterbatasan perangkat penulis hanya menggunakan 200 video dari dataset Celeb-DF dengan setiap video untuk dataset dengan hanya menggunakan 5 *frame* yang berisi wajah orang dari keseluruhan *frame* dalam setiap videonya.
4. Dataset yang digunakan untuk eksperimen memiliki dua kelas dengan rincian 1000 data wajah asli dan 1000 data wajah *deepfake* berformat ".jpg".
5. Pada penelitian ini tidak membahas mengenai deteksi objek.

1.3.2 Metode

1. Menggunakan *Data science methodology*, peneliti hanya menggunakan delapan dari sepuluh langkah yang ada yaitu tahap *business understanding* hingga *evaluation*, untuk tahap *deployment* dan *feedback* tidak diperlukan dalam penelitian ini, karena penelitian ini hanya sebatas pengujian. Berikut tahap yang digunakan : *Business understanding, Analytic approach, Data requirements, Data collection, Data understanding, Data preparation, Modeling, Evaluation*
2. Menggunakan Filter *Gaussian* untuk mengurangi kebisingan pada gambar.
3. Menggunakan *Local Binary Pattern* (LBP) untuk mendapatkan tekstur gambar.
4. Menggunakan MTCNN (*Multitask Cascaded Convolutional Networks*) untuk mengekstraksi wajah dari *frame*.
5. Model klasifikasi menggunakan model *XceptionNet* dan model *ResNet*-50 sebagai pembanding.

1.3.3 Tools

Berikut ini adalah *tool* yang penulis pergunakan, yaitu :

1. Pemodelan dilakukan menggunakan *Google Colab* dengan bahasa pemrograman *Python* dengan *library keras dan Tensorflow*.

2. Penelitian ini menggunakan notebook Asus, Windows 10 untuk pembuatan.
3. Penelitian ini menggunakan *Google Drive* sebagai tempat penyimpanan data.

1.4 Tujuan Penelitian

Penelitian ini bertujuan untuk menerapkan metode pemrosesan gambar filter Gaussian dan LBP dengan model *XceptionNet* dan model *ResNet*-50 sebagai pembanding untuk klasifikasi video *deepfake*.

1.5 Manfaat Penulisan

Adapun manfaat yang didapatkan adalah sebagai berikut :

1. **Bagi Penulis**
   - Menerapkan ilmu dan pengetahuan yang sudah didapat penulis dalam bidang pengolahan citra digital dan pembelajaran mesin.
   - Lebih memahami klasifikasi pembelajaran mesin
2. **Bagi Universitas**

   Menambah literatur tentang pengolahan citra digital dan pembelajaran mesin sebagai acuan untuk penelitian selanjutnya dan memperbarui dari penelitian yang sudah ada sebelumnya.
3. **Bagi Masyarakat**

   Membantu pihak terkait dalam menklasifikasi video *deepfake* untuk menghindari adanya kerugian.

1.6 Metodologi penelitian

Metodologi penelitian digunakan sebagai pedoman peneliti dalam pelaksanaan penelitian ini agar hasil yang dicapai sesuai dengan tujuan. Metodologi penelitian meliputi pengumpulan data hingga terbentuknya hasil akhir model klasifikasi video *deepfake*.

1.6.1 Metode Pengumpulan Data

Dalam melakukan penelitian ini, penulis menggunakan metode pengumpulan data, yaitu :

a. Studi Pustaka

Studi pustaka merupakan salah satu kegiatan pengumpulan data dan informasi dari jurnal, buku, dan lain-lain yang berkaitan sebagai sumber data dan informasi dalam penelitian yang dilakukan.

b. Metode Studi Lapangan

Penulis menggunakan metode obervasi melalui pengambilan video dari internet sebagai dataset uji terhadap metode klasifikasi video *deepfake* yang dibuat.

1.7 Sistematika Penulisan

Sistematika yang dibuat pada tugas akhir ini akan dibagi dalam enam bagian, yaitu :

BAB I PENDAHULUAN

Dalam bab ini membahas mengenai latar belakang penulisan, perumusan masalah, batasan masalah, tujuan dan manfaat, metode dan sistematika penulisan yang merupakan gambaran menyeluruh dari penulisan skripsi ini.

BAB II LANDASAN TEORI

Dalam bab ini membahas mengenai berbagai teori yang mendasari analisis permasalahan yang berhubungan dengan pembahasan.

BAB III METODOLOGI PENELITIAN

Bab ini berisi pembahasan atau pemaparan metode yang penulis pakai dalam pencarian data maupun perancangan sistem yang dilakukan pada penelitian.

BAB IV IMPLEMENTASI

Bab ini membahas mengenai implementasi dan pengujian sistem selama penelitian ini berlangsung.

BAB V HASIL DAN PEMBAHASAN

Bab ini membahas mengenai hasil dan pembahasan rancangan pembuatan klasifikasi video *deepfake* menggunakan pengolahan citra digital dan model *XceptionNet*.

BAB VI PENUTUP

Bab ini berisi kesimpulan dari hasil pembahasan seluruh bab serta saran-saran yang kiranya dapat diperhatikan serta dipertimbangkan untuk pengembangan sistem dimasa mendatang.

BAB II
LANDASAN TEORI

2.1 Klasifikasi Gambar

Klasifikasi adalah pengaturan sistematis dalam kelompok dan kategori berdasarkan fitur-fiturnya. Klasifikasi gambar ada untuk mengurangi kesenjangan antara *computer vision* dan *human vision* dengan melatih komputer dengan data. Klasifikasi gambar dilakukan dengan membedakan gambar ke dalam kategori yang ditentukan berdasarkan fitur (Krishna et al., 2018).

Menurut Aggarwal & Kaur (2018) tujuan dari klasifikasi gambar adalah penggolongan gambar yang diprogram untuk kelas. Dua jenis klasifikasi yaitu, *supervised classification* dan *unsupervised classification*. Proses klasifikasi gambar meliputi dua langkah berupa pelatihan kemudian pengujian. Proses pelatihan untuk mengambil sifat karakteristik gambar dan membentuk deskripsi khusus untuk kelas tertentu. Langkah pengujian untuk mengklasifikasikan gambar uji dengan sistem yang telah dilatih.

2.2 Deepfake

Deepfake merupakan bagian dari *deep learning* untuk mengenali dan bertukar wajah dalam gambar dan video. Teknik ini dimulai dengan menganalisis sejumlah besar foto atau video wajah seseorang, melatih algoritma kecerdasan buatan untuk memanipulasi wajah tersebut, dan kemudian menggunakan algoritma untuk memetakan wajah tersebut ke seseorang dalam sebuah video (Dodge & Johnstone, 2018).

Menurut Spivak (2019) istilah *deepfake* merujuk pada video yang melapisi wajah hiper-*real*istis ke tubuh orang lain dengan maksud membuat video baru dengan representasi palsu. Patrini (2018) menyatakan *deepfake* merujuk pada konten audio visual foto-*real*istis yang dihasilkan dengan bantuan pembelajaran mendalam. Istilah ini berasal dari pengguna "*deepfakes*", Reddit merupakan yang

pertama secara publik mendokumentasikan upaya untuk secara sintetis mengganti wajah orang yang menjadi target dengan wajah orang lain.

2.3 Dataset

Menurut Putra (2019) Dataset adalah kumpulan data (sampel dalam statistik). Sampel ini adalah data yang digunakan untuk membuat model maupun mengevaluasi model *machine learning*. Umumnya, dataset dibagi menjadi tiga jenis yang tidak beririsan (suatu sampel pada himpunan tertentu tidak muncul pada himpunan lainnya):

1. *Training set* adalah himpunan data yang digunakan untuk melatih atau membangun model.
2. *Development set* atau *validation set* adalah himpunan data yang digunakan untuk mengoptimisasi saat melatih model. Model dilatih menggunakan *training set* dan pada umumnya kinerja saat latihan diuji dengan *validation set*. Hal ini berguna untuk generalisasi (agar model mampu mengenali pola secara generik).
3. *Testing set* adalah himpunan data yang digunakan untuk menguji model setelah proses latihan selesai. Istilah *testing dataset* atau *test set* mengacu pada *testing set*. *testing set* adalah *unseen* data. Artinya, model tidak boleh melihat sampel ini saat proses latihan.

2.3.1 Celeb-DF(V2)

Dataset Celeb-DF(V2) adalah dataset buatan Li (2019) yang terdiri dari 590 video nyata dan 5639 video *Deepfake*.

Gambar 2.1 : Contoh Dataset Celeb-DF(V2) dengan Kotak Hijau merupakan wajah asli dan kotak merah merupakan wajah *deepfake*

Sumber : (Li et al., 2020)

Dataset	# Real		# DeepFake		Release Date
	Video	Frame	Video	Frame	
UADFV	49	17.3k	49	17.3k	2018.11
DF-TIMIT-LQ DF-TIMIT-HQ	320*	34.0k	320 320	34.0k 34.0k	2018.12
FF-DF	1,000	509.9k	1,000	509.9k	2019.01
DFD	363	315.4k	3,068	2,242.7k	2019.09
DFDC	1,131	488.4k	4,113	1,783.3k	2019.10
Celeb-DF	590	225.4k	**5,639**	2,116.8k	2019.11

Gambar 2.2 : Informasi Dasar Dataset Celeb-DF(V2)

Sumber : (Li et al., 2020)

Durasi rata-rata semua video adalah 13 detik dengan *frame rate* standar 30 *frame* per detik. Video asli diambil dari video YouTube, berisi wawancara 59 selebriti dengan distribusi yang beragam, dalam jenis kelamin, usia, dan kelompok etnis. 56,8% subjek dalam video nyata adalah laki-laki, dan 43,2% adalah perempuan. 8,5% berusia 60 ke atas, 30,5% usia antara 50 - 60, 26,6% adalah usia 40-an, 28,0% berusia 30-an, dan 6,4% lebih muda dari usia 30. 5,1% adalah orang Asia, 6,8% adalah Afrika Amerika dan 88,1%

adalah Kaukasia. Video *deepfake* dibuat dengan menukar wajah untuk setiap pasangan dari 59 subjek dengan hasil video akhir dalam format MPEG4.0 (Li et al., 2019).

2.4 Pengolahan Citra Digital

Menurut Kaur & Kaur (2015) pengolahan citra digital adalah mengubah citra menjadi versi modifikasi yang lebih baik. Dalam ilmu komputer, pemrosesan gambar adalah segala bentuk pemrosesan sinyal yang inputnya berupa gambar atau bingkai video dan outputnya dapat berupa gambar atau kumpulan parameter yang terkait dengan gambar. Pengolahan citra adalah proses perbaikan citra atau fitur-fiturnya untuk mendapatkan detail yang maksimal dan menonjolkan informasi yang bermanfaat. Aplikasi dasar pengolahan citra adalah:

- Meningkatkan kualitas visual gambar ke tingkat berikutnya.
- Mempersiapkan gambar untuk ekstraksi fitur semaksimal mungkin.

Menurut Ravi & Ashokkumar (2017) Pengolahan citra digital mengacu pada pengolahan gambar dua dimensi oleh komputer digital. Citra digital adalah larik bilangan aktual yang diwakili oleh jumlah bit yang terbatas. Keuntungan prinsip dari metode *Digital Image Processing* adalah keserbagunaannya, pengulangan dan pelestarian presisi data asli. Klasifikasi teknik Pengolahan Citra adalah :

- *Image representation* (Representasi citra)
- *Image preprocessing* (Praproses citra)
- *Image enhancement* (Peningkatan citra)
- *Image analysis* (Analisis citra)
- *Image compression* (Kompresi citra)

2.4.1 Image Enhancement

Image enhancement atau peningkatan citra pada dasarnya meningkatkan interpretasi atau persepsi informasi dalam gambar untuk manusia dan memberikan masukan `lebih baik' untuk teknik pemrosesan

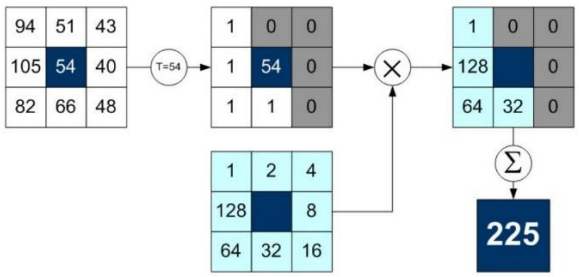
gambar otomatis lainnya. Motif utama di balik peningkatan citra adalah untuk memodifikasi atribut dari citra yang diberikan agar sesuai dengan tugas yang pengamat diberikan. Proses modifikasi dapat bervariasi sesuai dengan tugas yang diberikan (Kaur & Kaur, 2015).

Menurut Ravi & Ashokkumar (2017) Teknik peningkatan gambar dapat dibagi menjadi dua kategori luas, yaitu :

- Metode domain spasial, yang berfungsi langsung pada piksel
- Metode domain frekuensi, yang berfungsi pada transformasi Fourier suatu citra.

2.4.2 Local Binary Pattern

Menurut Prakasa (2016) *Local binary pattern* adalah operator untuk deskripsi tekstur yang berdasarkan tanda-tanda perbedaan antara piksel tetangga dan piksel pusat. Untuk setiap nilai piksel pada citra, diperoleh kode biner dengan *thresholding* lingkungannya dengan nilai piksel tengah. Kode biner ini dapat dianggap sebagai pola biner. Piksel tetangga menjadi 1 jika nilai piksel lebih besar dari atau sama dengan nilai ambang batas, dan menjadi 0 jika nilai piksel kurang dari ambang batas.



Gambar 2.3 : Tahapan Perhitungan LBP

Sumber : (Prakasa, 2016)

LBP didapatkan dengan menerapkan persamaan berikut :

$$LBP_{P,R} = \sum_{p=0}^{P-1} s(g_p - g_c)2^p,\ s(x) = \begin{cases} 1, x \geq 0 \\ 0, x < 0 \end{cases}$$

2.4.3 Filter Gaussian

Gaussian blur (juga dikenal sebagai pemulusan Gaussian) adalah efek samping dari mengaburkan gambar dengan fungsi Gaussian. Ini adalah efek yang umum digunakan dalam pemrograman ilustrasi, secara teratur untuk mengurangi noise gambar dan mengurangi detail. Visualisasi yang disempurnakan dari prosedur yang mengaburkan ini adalah kabut halus yang berkurang (Padmaja et al., 2020).

Menurut Vyas (2018) Filter Gaussian sangat penting baik untuk alasan teoritis maupun praktis. Filter gambar menggunakan kernel diskrit yang diturunkan dari bentuk simetris radial dari fungsi Gaussian 2-D kontinyu didefinisikan sebagai berikut :

$$f(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$$

Perkiraan diskrit untuk fungsi kontinu diatas ditentukan menggunakan dua parameter :

- ukuran kernel yang diinginkan (sebagai masker filter N x N)
- nilai σ, standar deviasi dari fungsi Gaussian.

2.5 Kecerdasan Buatan

Artificial Intelligence (AI) adalah sebutan yang umum digunakan untuk bidang ilmu yang menerapkan kecerdasan manusia ke dalam mesin. Bertujuan menyediakan mesin dengan kapasitas melakukan fungsi seperti logika, penalaran, perencanaan, pembelajaran, dan persepsi (Perez et al., 2018).

I Made Suandi Putra MSc (2017) berpendapat bahwa AI dapat dipandang sebagai kemajuan signifikan terbaru dalam rangkaian kemajuan yang telah terjadi dalam perkembangan teknologi. Apa yang menjadi perkembangan terbaru adalah

kemajuan dan skalabilitas teknologi yang membuat penerapan AI dapat dipergunakan secara praktis.

Menurut Wisskirchen (2017) Ada dua jenis kecerdasan buatan:

- Kecerdasan buatan yang lemah: Komputer hanyalah alat untuk menyelidiki proses kognitif - komputer mensimulasikan kecerdasan. Dalam hal ini komputer diberi data oleh pengguna.
- Kecerdasan buatan yang kuat: Proses di komputer adalah proses belajar mandiri intelektual. Komputer dapat 'memahami' dengan menggunakan perangkat lunak / pemrograman yang tepat dan mampu mengoptimalkan perilaku mereka sendiri berdasarkan perilaku dan pengalaman mereka sebelumnya. Dalam hal ini pengguna tidak perlu memberikan data kepada komputer.

2.5.1 Machine Learning

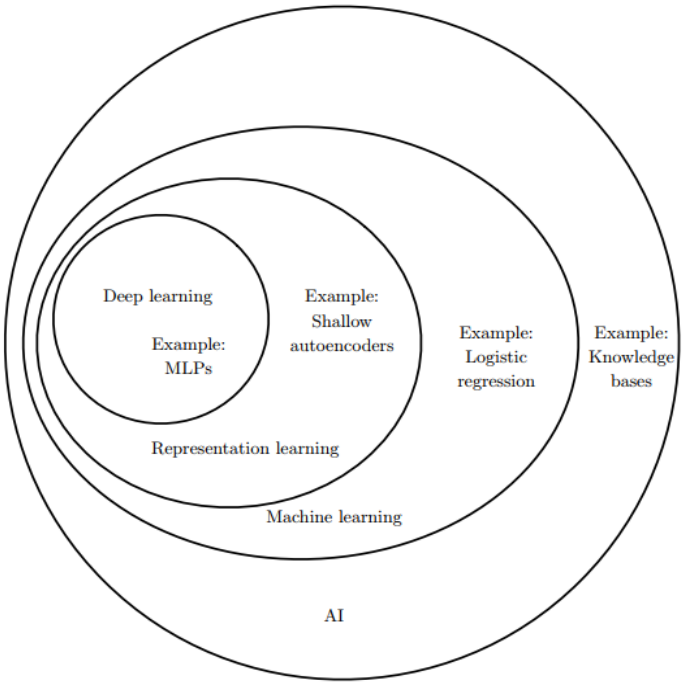
Machine learning merupakan salah satu bagian dari kecerdasan buatan. Menurut Putra (2019) *machine learning* adalah teknik untuk melakukan inferensi terhadap data dengan pendekatan matematis. Inti *machine learning* adalah untuk membuat model (matematis) yang merefleksikan pola-pola data.

Machine Learning atau pembelajaran mesin merupakan bagian dalam kecerdasan buatan yang dilatih untuk dapat menggantikan atau menirukan perilaku manusia untuk menyelesaikan masalah atau melakukan otomatisasi. Setidaknya ada dua aplikasi utama dalam *machine learning* yaitu, klasifikasi dan prediksi. Ciri khas dari *machine learning* adalah adanya proses pelatihan, pembelajaran, atau training. Oleh karena itu, *machine learning* membutuhkan data untuk dipelajari yang disebut sebagai data training (Ahmad, 2017).

2.5.2 Deep Learning

Menurut Kim (2016) *deep learning* adalah bentuk pembelajaran mesin yang memungkinkan komputer untuk belajar dari pengalaman dan memahami dunia dalam hal hierarki konsep.

Metode *deep learning* adalah representasi metode pembelajaran dengan beberapa tingkatan representasi, diperoleh dengan menyusun modul-modul yang sederhana tetapi *non linier* yang masing-masing mengubah representasi pada satu level (dimulai dengan input) menjadi representasi di tingkat yang lebih tinggi, sedikit level yang lebih abstrak.(Lecun et al., 2015)



Gambar 2.4 : Diagram Deep Learning

Sumber : (Goodfellow, 2016)

Deep learning adalah sejenis pembelajaran representasi, yang pada gilirannya merupakan sejenis *machine learning*, yang digunakan untuk banyak pendekatan, tetapi tidak semua ke AI. Setiap bagian dari diagram venn menyertakan contoh teknologi AI (Goodfellow, 2016).

2.6 Tensorflow

Tensorflow adalah antarmuka untuk mengekspresikan algoritma pembelajaran mesin, dan implementasi untuk mengeksekusi algoritma tersebut. Komputasi yang diekspresikan menggunakan Tensorflow dapat dijalankan dengan sedikit atau tanpa perubahan pada beragam sistem heterogen, mulai dari perangkat seluler seperti ponsel dan tablet hingga sistem terdistribusi berskala besar dari ratusan mesin dan ribuan perangkat komputasi seperti kartu GPU (Abadi et al., 2016).

Menurut Tutorialspoint (2019) Tensorflow adalah *framework machine learning open source* untuk semua *developer*. Tensorflow digunakan untuk mengimplementasikan pembelajaran mesin dan aplikasi pembelajaran mendalam, untuk mengembangkan dan meneliti ide-ide menarik tentang kecerdasan buatan, tensorflow dibuat oleh tim Google. Tensorflow dirancang dalam bahasa pemrograman Python. Berikut beberapa fitur penting TensorFlow :

a. Mencakup fitur yang mendefinisikan, mengoptimalkan dan menghitung matematika ekspresi dengan mudah dengan bantuan array multi-dimensi yang disebut tensor.
b. Mencakup dukungan pemrograman *deep neural networks* dan teknik pembelajaran mesin.
c. Mencakup fitur komputasi skalabel tinggi dengan berbagai kumpulan data.
d. TensorFlow menggunakan komputasi GPU, mengotomatiskan pengelolaan. Juga termasuk sebuah fitur unik dari pengoptimalan memori yang sama dan data yang digunakan.

2.7 Convolutional Neural Network

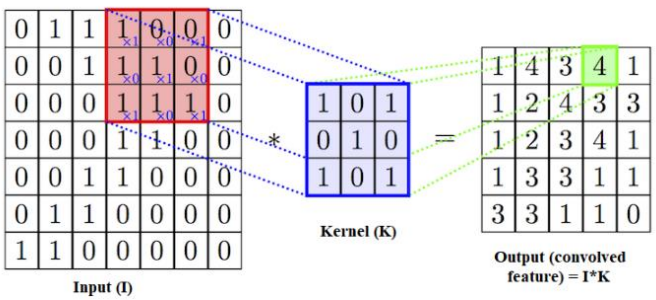
Convolutional Neural Network (CNN atau ConvNet) adalah sebuah jaringan saraf multi-layer yang terinspirasi oleh mekanisme sistem optik makhluk hidup (Sultana et al., 2018). Convolutional Neural Network (CNN) adalah pengembangan dari Multilayer Perceptron (MLP) yang didesain untuk mengolah data dua dimensi.

CNN termasuk dalam jenis Deep Neural Network karena kedalaman jaringan yang tinggi dan banyak diaplikasikan pada data citra (P et al., 2016).

CNN memiliki kemampuan untuk secara otomatis menemukan fitur kontekstual yang relevan dalam masalah kategorisasi gambar. CNN terdiri dari setumpuk filter konvolusi yang mengekstrak fitur gambar kontekstual hierarkis, dan merupakan bentuk populer dari *deep learning networks* (Maggiori et al., 2016). Sebuah CNN tradisional terdiri dari satu atau beberapa blok convolutional dan *pooling layer*, diikuti oleh satu atau beberapa *fully connected layer* dan sebuah *output layer* (Ghosh et al., 2020).

2.7.1 Convolutional Layer

Lapisan konvolusi terdiri dari beberapa kernel atau filter konvolusi yang dapat belajar, yang digunakan untuk menghitung peta fitur yang berbeda. Setiap unit peta fitur terhubung ke bidang reseptif di lapisan sebelumnya. Peta fitur baru dihasilkan dengan menggabungkan input dengan kernel dan menerapkan fungsi aktivasi elemen non-linear pada hasil yang dikonvolusikan.



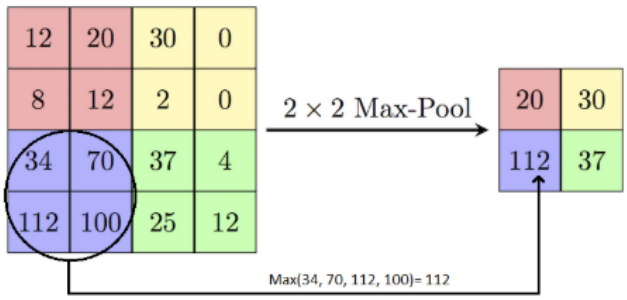
Gambar 2.5 : Lapisan Konvolusi

Sumber : (Sultana et al., 2018)

2.7.2 Pooling Layer

Pooling atau *sub-sampling layer* mengambil wilayah kecil dari *output convolutional* sebagai *input* dan menurunkannya untuk menghasilkan *output* tunggal. Ada beberapa teknik *sub-sampling* yang

berbeda sebagai contoh max pooling, min pooling, average pooling, dll. Pooling mengurangi jumlah parameter yang akan dihitung serta membuat terjemahan jaringan menjadi invariant (Ghosh et al., 2020).

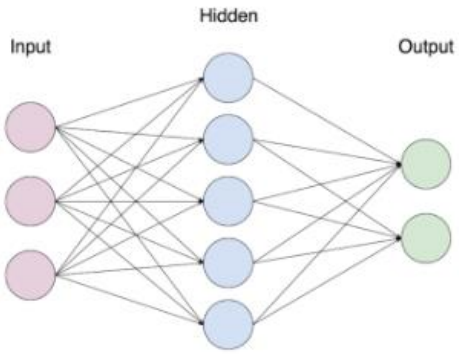


Gambar 2.6 : Pooling Layer

Sumber : (Sultana et al., 2018)

2.7.3 Fully Connected Layer

Lapisan *Fully Connected* (FC) mengambil input dari *pooling* akhir atau lapisan konvolusi dan menghasilkan output akhir CNN. Dalam hal klasifikasi citra, CNN dapat dilihat sebagai kombinasi dari dua bagian: bagian ekstraksi ciri dan bagian klasifikasi. *Convolution* dan *pooling layer* melakukan ekstraksi fitur. Di atas fitur ini, lapisan FC ditambahkan sebagai pengklasifikasi (Ghosh et al., 2020).

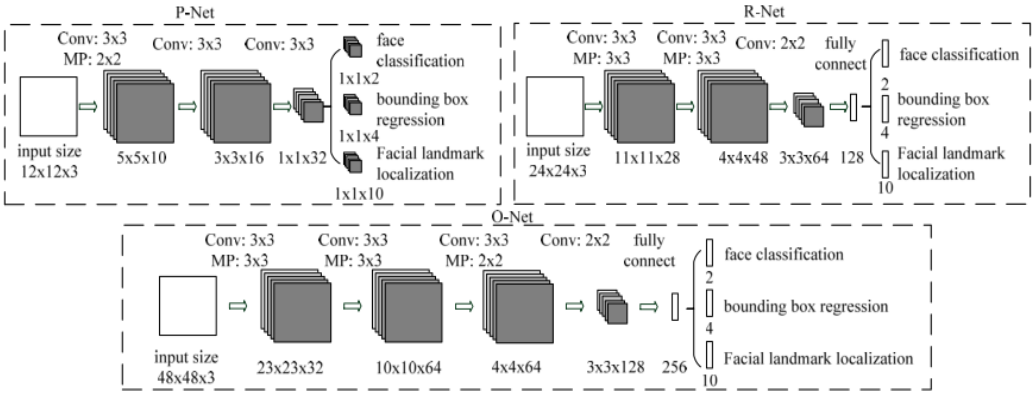


Gambar 2.7 : Fully Connected Layer

Sumber : (Sultana et al., 2018)

2.8 MTCNN

Menurut Montserrat (2020) MTCNN (*Multitask Cascaded Convolutional Networks*) adalah model multi-tugas bertingkat yang dapat menghasilkan kotak pembatas wajah dan tanda wajah secara bersamaan. Model ini menggunakan arsitektur tiga tahap bertingkat dalam memprediksi lokasi wajah dan tanda wajah.



Gambar 2.8 : Arsitektur MTCNN

Sumber : (K. Zhang et al., 2016)

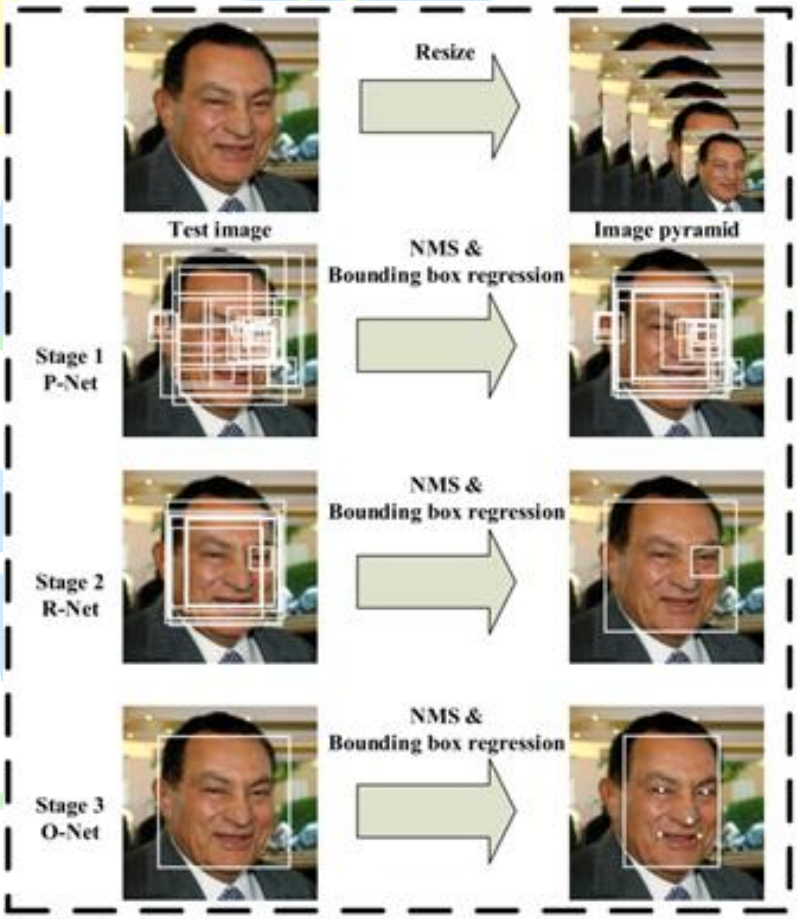
Kerangka tema MTCNN serupa dengan cascade *Convolutional Neural Networks* (CNN), tetapi berhubungan dengan deteksi area wajah dan deteksi fitur wajah secara bersamaan. Sama dengan banyak model CNN lain yang menargetkan untuk mengatasi masalah gambar, MTCNN juga menerapkan gambar piramida, regresi kotak pembatas *Non Maximum Suppression* (NMS) dan rangkaian teknologi CNN (Du, 2020).

Menurut K. Zhang (2016) Langkah-langkah dari MTCNN ditunjukkan pada Gambar 2.5. Diberikan sebuah gambar, dan selanjutnya mengubah ukurannya ke skala yang berbeda untuk membangun piramida gambar, yang merupakan masukan dari kerangka cascade tiga tahap berikut :

- Tahap 1 : mengeksploitasi fully convolutional network, yang disebut *Proposal Network* (P-Net), untuk mendapatkan jendela wajah dan vektor regresi kotak pembatasnya. Kemudian kandidat dikalibrasi berdasarkan

estimasi vektor regresi kotak pembatas. Setelah itu, menggunakan NMS untuk menggabungkan kandidat yang tumpang tindih.

- Tahap 2 : Semua kandidat diumpankan ke CNN lain, yang disebut *Refine Network* (R-Net), yang selanjutnya menolak sejumlah besar kandidat palsu, melakukan kalibrasi dengan regresi kotak pembatas, dan melakukan NMS.
- Tahap 3 : Tahap ini mirip dengan tahap kedua, tetapi dalam hal ini bertujuan untuk mengidentifikasi daerah wajah dengan lebih banyak pengawasan. Secara khusus, jaringan akan menampilkan lima posisi *landmark* wajah.



Gambar 2.9 : Langkah-langkah dari MTCNN

Sumber : (K. Zhang et al., 2016)

2.9 Keras

Menurut Chollet (2015) Keras adalah API tingkat tinggi dari TensorFlow 2, merupakan antarmuka produktif dan mudah digunakan untuk memecahkan masalah pembelajaran mesin, dengan fokus pada pembelajaran mendalam modern.

Menurut Tutorialspoint (2019) Keras adalah library Python tingkat tinggi yang ringkas, mudah dipelajari, dan dijalankan di atas TensorFlow kerangka. Keras dibuat dengan fokus pada pemahaman teknik pembelajaran yang mendalam, seperti membuat lapisan untuk jaringan saraf mempertahankan konsep bentuk dan detail matematika. Delapan langkah berikut untuk membuat model pembelajaran dalam di Keras :

a. Memuat data
b. Memproses ulang data yang dimuat
c. Definisi model
d. Menyusun model
e. Sesuaikan model yang ditentukan
f. Mengevaluasi
g. Buat prediksi yang diperlukan
h. Simpan model

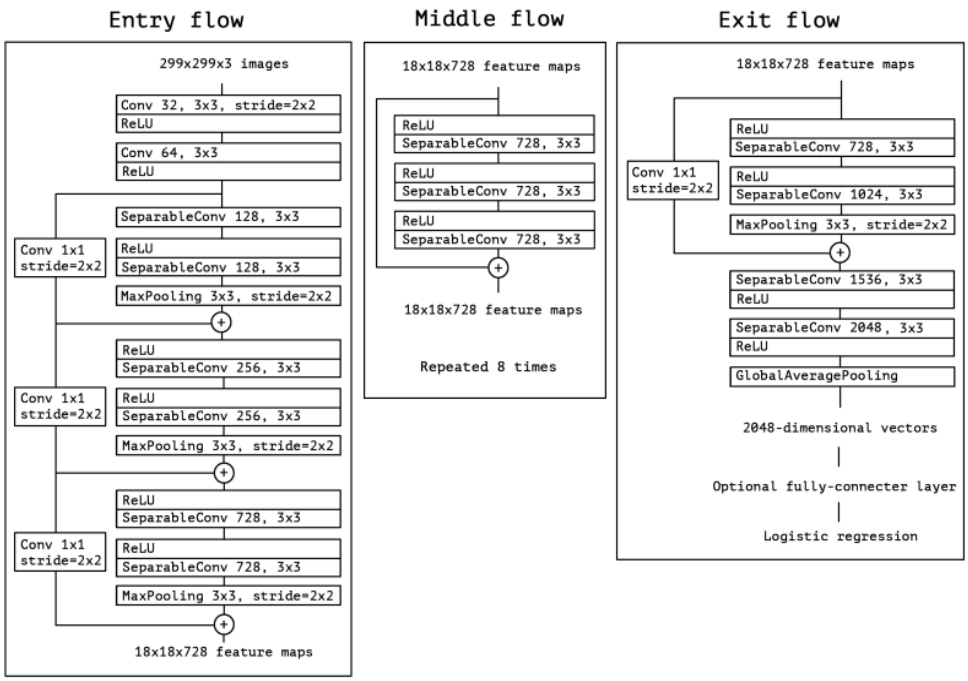
2.9.1 *XceptionNet*

Menurut Chollet (2016) *XceptionNet* merupakan jaringan saraf konvolusional dalam baru yang terinspirasi oleh Inception, di mana modul Inception telah diganti dengan separable konvolusi, arsitektur ini disebut sedikit berkinerja lebih baik dari Inception V3 pada dataset ImageNet (yang dirancang untuk Inception V3), dan secara signifikan mengungguli Inception V3 pada dataset klasifikasi gambar yang lebih besar yang terdiri dari 350 juta gambar dan 17.000 kelas.

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
Xception	88	0.790	0.945	22,910,480	126	109.42	8.06

Gambar 2.10 : spesifikasi Model *XceptionNet*

Arsitektur *XceptionNet* memilki tahap pertama melewati aliran *entry*, kemudian melalui aliran tengah yang diulang delapan kali, dan akhirnya melalui aliran keluar, seperti pada gambar berikut :



Gambar 2.11 : Arsitektur Model *XceptionNet*

Sumber : (Chollet, 2016)

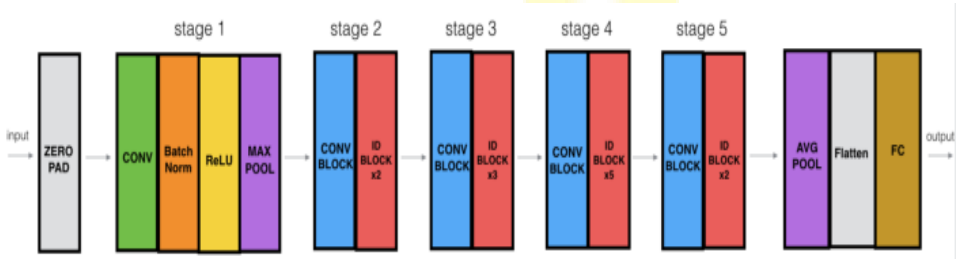
2.9.2 *ResNet*-50

Menurut Akbar (2020) *ResNet*-50 adalah salah satu varian *ResNet* yang memiliki 50 *layer*. Jika pada varian *ResNet* sebelumnya dilakukan *skip connection* sebanyak 2 *layer*, maka *ResNet*-50 melewati 3 *layer* dan terdapat 1x1 *convolution layer*.

Model	Size (MB)	Top-1 Accuracy	Top-5 Accuracy	Parameters	Depth	Time (ms) per inference step (CPU)	Time (ms) per inference step (GPU)
ResNet50	98	0.749	0.921	25,636,712	-	58.20	4.55

Gambar 2.12 : Spesifikasi Model *ResNet*-50

Jaringan Residual (*ResNets*) *deep convolutional networks* ide dasarnya adalah untuk melewati blok lapisan konvolusi dengan menggunakan koneksi pintasan. Blok dasar bernama "*bottleneck*" mengikuti dua aturan desain sederhana, yaitu : untuk ukuran peta fitur keluaran yang sama, lapisan memiliki jumlah filter yang sama, dan jika ukuran peta fitur dibelah dua, jumlah filter menjadi dua kali lipat (Rezende et al., 2017).



Gambar 2.13 : Arsitektur Model *ResNet*-50

2.10 Area Dibawah Kurva (AUC)

Menurut Google-developers (2020) sebuah kurva ROC (*Receiver Operating Characteristic Curve*) adalah grafik yang menunjukkan kinerja model klasifikasi pada seluruh ambang klasifikasi. Kurva ini menggambarkan dua parameter :

- Tingkat Positif Benar
- Tingkat Positif Palsu

$$F_{PR} = \frac{F_{TP}}{T_{TN}+F_{FP}}$$

$$T_{PR} = \frac{T_{TP}}{F_{FN}+T_{TP}}$$

Kurva ROC menggambarkan TPR vs. FPR pada batas klasifikasi yang berbeda. Menurunkan batas klasifikasi akan mengklasifikasikan lebih banyak item sebagai positif, sehingga meningkatkan Positif Palsu dan Positif Benar Untuk menghitung titik-titik dalam kurva ROC, dapat dengan mengevaluasi model regresi logistik berkali-kali dengan batas klasifikasi yang berbeda, tetapi hal ini akan menjadi tidak efisien. Untuk itu, ada algoritma berbasis pengurutan yang efisien

yang dapat memberikan informasi ini yang disebut AUC (Google- developers, 2020).

Menurut Yang & Berdine (2017) AUC (juga dikenal sebagai c-statistic) dapat digunakan untuk mengevaluasi kemampuan diagnostik dari suatu tes untuk membedakan status yang sebenarnya. Secara umum, aturan praktis untuk menafsirkan nilai AUC adalah:

AUC=0.5	No discrimination, e.g., randomly flip a coin
0.6≥AUC>0.5	Poor discrimination
0.7≥AUC>0.6	Acceptable discrimination
0.8≥AUC>0.7	Excellent discrimination
AUC>0.9	Outstanding discrimination

Gambar 2.14 : Aturan Penafsiran Nilai AUC

Sumber : (Yang & Berdine, 2017)

Kurva ROC dapat dengan baik mewakili generalisasi kinerja sebuah model. AUC adalah area di bawah kurva ROC. Semakin besar nilai AUC, semakin baik kinerjanya. Rumus perhitungan AUC adalah sebagai berikut, dengan m adalah jumlah sampel :

$$A_{AUC} = \frac{1}{2}\sum_{i=1}^{m}\left(F_{PR}^{(i+1)} - F_{PR}^{(i)}\right) \times \left(T_{PR}^{(i)} - T_{PR}^{(i+1)}\right)$$

Area di bawah kurva ROC (AUC) bervariasi antara 0,5 (untuk garis diagonal 45 ° yang mewakili tes yang tidak informatif) dan 1,0 untuk uji yang sempurna. AUC khususnya berguna ketika dua atau lebih tes diagnostik dibandingkan. Memiliki AUC yang lebih tinggi, tes dengan ROC kurva yang sepenuhnya berada di atas kurva lain, adalah jelas lebih baik (Habibzadeh et al., 2016).

2.11 Confusion Matrix

Menurut Yang & Berdine (2017) *Confusion Matrix* (juga dikenal sebagai matriks kesalahan) adalah tabel kontingensi yang digunakan untuk mendeskripsikan kinerja sistem klasifikasi. Untuk klasifikasi biner, *Confusion Matrix* direpresentasikan sebagai matriks 2x2. Sebuah *Confusion Matrix*, memiliki empat ukuran, yaitu, positif benar (TP), Negatif benar (TN), positif palsu (FP), dan negatif palsu (FN), untuk dilaporkan (Hasnain et al., 2020).

		Predicted condition	
		Disease	**Normal**
True condition	Disease	True positive (TP) (sensitivity)	False negative (FN)
	Normal	False positive (FP)	True negative (TN) (specificity)

Gambar 2.15 : Contoh Confusion Matrix

Sumber : (Yang & Berdine, 2017)

Confusion Matrix berisi semua informasi mentah tentang prediksi yang dilakukan oleh sebuah model klasifikasi pada kumpulan data tertentu. Untuk mengevaluasi akurasi generalisasi model, adalah umum untuk menggunakan kumpulan data pengujian yang tidak digunakan selama proses pembelajaran model tersebut (Caelen, 2017).

2.12 F1 Score

Menurut Vakili et al. (2020) metrik yang juga dikenal sebagai f-score atau f-measure, mengambil presisi dan *recall* ke dalam pertimbangan untuk menghitung kinerja dari sebuah algoritma. Secara matematis, itu adalah harmonic rata-rata presisi dan *recall* yang dirumuskan sebagai berikut :

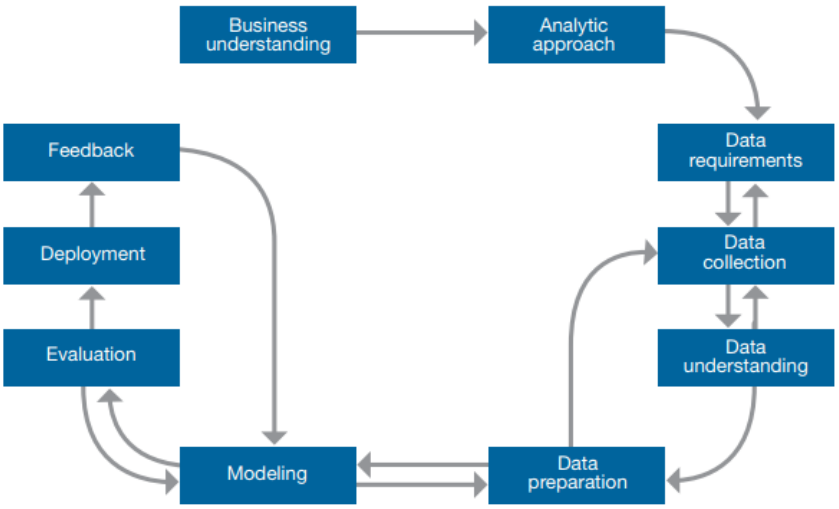
$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.13 Google Colaboratory

Google Colaboratory atau akrab disebut dengan *Google Colab* merupakan sebuah aplikasi pengolahan data berbasis *cloud* dan *free* yang disediakan untuk tujuan penelitian atau proses pembelajaran. *Tools* ini dibuat dengan *environment* jupyter dan mendukung hampir semua *library* yang diperlukan dalam proses *development* AI atau *Artificial Intelligence* (Wahyu, 2020).

Notebook *Colaboratory* didasarkan pada Jupyter, jupyter adalah *open-source* dan alat berbasis browser yang mengintegrasikan bahasa yang ditafsirkan, perpustakaan, dan alat untuk visualisasi. *Colaboratory* berfungsi sebagai objek Google Dokumen yang dapat dibagikan dan pengguna dapat berkolaborasi pada notebook yang sama. *Colaboratory* menyediakan runtime Python 2 dan 3 yang telah dikonfigurasi sebelumnya dengan pustaka pembelajaran mesin esensial dan kecerdasan buatan, seperti TensorFlow, Matplotlib, dan Keras. Serta infrastruktur Google Colaboratory dihosting di platform Google Cloud (Carneiro et al., 2018).

2.14 Data Science Methodology



Gambar 2.16 : Diagram Data Science Methodology

Sumber : (John B. Rollins, 2015)

Menurut John B. Rollins (2015), *Data Science Methodology* terdiri dari 10 tahap yang membentuk iterative proses menggunakan data untuk mengungkap wawasan, dengan tahapan sebagai berikut :

1. *Business understanding*

   Tahap ini merupakan tahap pertama meletakkan dasar untuk resolusi yang sukses dari masalah bisnis. Untuk membantu menjamin keberhasilan proyek, dan memastikan pekerjaan tetap pada jalurnya untuk menghasilkan solusi yang diinginkan.

2. *Analytic approach*

   Setelah masalah bisnis dinyatakan dengan jelas, dapat ditentukan pendekatan analitik untuk memecahkan masalah. Tahap ini memerlukan pengungkapan masalah dalam konteks teknik statistik dan pembelajaran mesin, sehingga organisasi dapat mengidentifikasi yang paling cocok untuk hasil yang diinginkan

3. *Data requirements*

   Pendekatan analitik yang dipilih menentukan persyaratan dan kebutuhan data. Secara khusus, metode analitik yang akan digunakan memerlukan konten, format, dan representasi data tertentu, dipandu oleh pengetahuan domain

4. *Data collection*

   Pada tahap pengumpulan data awal, ilmuwan data mengidentifikasi dan mengumpulkan sumber daya data yang tersedia—terstruktur, tidak terstruktur dan semi-terstruktur—relevan dengan domain masalah. Jika ada celah di pengumpulan data, ilmuwan data mungkin harus merevisi persyaratan dan kebutuhan data yang sesuai dan mengumpulkan data baru dan/atau lebih.

5. *Data understanding*

   Setelah pengumpulan data, para ilmuwan data biasanya menggunakan statistik deskriptif dan teknik visualisasi untuk memahami konten data, menilai kualitas data, dan menemukan wawasan awal tentang data. Pengumpulan data tambahan mungkin diperlukan untuk mengisi kesenjangan.

6. *Data preparation*

   Tahap ini mencakup semua kegiatan untuk mengkonstruksi dataset yang akan digunakan pada tahap pemodelan selanjutnya. Data kegiatan persiapan meliputi pembersihan data (menangani nilai yang hilang atau tidak valid, menghilangkan duplikat, memformat dengan benar), menggabungkan data dari berbagai sumber (file, tabel, platform) dan mengubah data menjadi variabel yang lebih berguna.

7. *Modeling*

   Dimulai dengan versi pertama dari kumpulan data yang disiapkan, tahap pemodelan berfokus pada pengembangan prediktif atau deskriptif model sesuai dengan pendekatan analitik yang ditentukan sebelumnya. Dengan model prediktif, ilmuwan data menggunakan set pelatihan (data historis di mana hasil yang diinginkan diketahui) untuk membangun modelnya.

8. *Evaluation*

   Selama pengembangan model dan sebelum penerapan, ilmuwan data mengevaluasi model untuk memahami kualitasnya dan memastikan bahwa itu benar dan sepenuhnya menangani bisnis masalah. Evaluasi model memerlukan komputasi berbagai tindakan diagnostik dan keluaran lainnya seperti tabel dan tables grafik, memungkinkan ilmuwan data untuk menafsirkan model kualitas dan kemanjurannya dalam memecahkan masalah. Untuk prediksi model, ilmuwan data menggunakan set pengujian, yang independen dari set pelatihan tetapi mengikuti distribusi probabilitas yang sama dan memiliki hasil yang diketahui. Set pengujian digunakan untuk mengevaluasi model sehingga dapat disempurnakan sesuai kebutuhan. Terkadang final model diterapkan juga untuk set validasi untuk penilaian akhir.

9. *Deployment*

   Setelah model yang memuaskan telah dikembangkan dan disetujui oleh sponsor bisnis, model dikerahkan ke dalam produksi lingkungan pengujian atau lingkungan pengujian yang sebanding. Biasanya itu dikerahkan secara terbatas sampai kinerjanya telah sepenuhnya dievaluasi.

10. *Feedback*

Dengan mengumpulkan hasil dari model yang diimplementasikan, organisasi mendapat umpan balik tentang kinerja model dan dampaknya terhadap lingkungan di mana ditempatkan. Menganalisa umpan balik ini memungkinkan ilmuwan data untuk menyempurnakan model untuk meningkatkan akurasi dan kegunaannya.

2.15 Penelitian Terkait

Penulis mengumpulkan data dan informasi dari penelitian terdahulu yang serupa untuk mendapatkan teori, kelebihan dan kekurangan penelitian-penelitian tersebut. Pada penelitian yang dilakukan oleh Rossler (2019) yang berjudul "FaceForensics++: *Learning to Detect Manipulated Facial Images*" menjelaskan tentang deteksi wajah palsu berbasis pembelajaran mesin menggunakan model *XceptionNet*. Peneltian selanjutnya oleh Boulkenafet (2015) dengan judul "*Face Anti-Spoofing Based On Color Texture Analysis*" menjelaskan tentang mendeteksi wajah palsu dengan menggunakan metode ekstraksi tekstur *Local binary pattern* (LBP) dengan penggolong *Support Vector Machine* (SVM) pada ruang warna YCbCr+HSV.

Penelitian oleh Kumar (2020) dengan judul "*Detecting Deepfakes with Metric Learning*" menjelaskan tentang pendekatan pembelajaran mendalam dalam konteks deteksi *deepfake* dalam skenario kompresi tinggi dan menunjukkan bahwa pendekatan yang diusulkan berdasarkan pembelajaran metrik menggunakan arsitektur jaringan triplet terbukti bisa sangat efektif dalam melakukan deteksi *deepfake*. Peneltian selanjutnya oleh Siddiqui (2016) dengan judul "*Face anti-spoofing with multifeature videolet aggregation*" menjelaskan tentang deteksi wajah palsu menggunakan metode ekstraksi tekstur multi-LBP and *Histogram of Oriented Optical Flow* (HOOF) dengan SVM.

Penelitian selanjutnya Dang (2020) dengan judul "*On the Detection of Digital Face Manipulation*" menjelaskan tentang deteksi wajah palsu menggunakan

Manipulation Appearance Model And Direct Regression dengan model *XceptionNet*. Penelitian oleh Khalil (2021) dengan judul "iCaps-Dfake: *An Integrated Capsule-Based Model for Deepfake Image and Video Detection*" menjelaskan tentang deteksi video *deepfake* dengan menggabungkan dua metode ekstraksi fitur, yaitu : *Local Binary Patterns* berbasis tekstur dan *Convolutional Neural Networks* (CNN) berbasis Modifikasi *High-Resolution Network* (HRNet), dengan aplikasi jaringan saraf kapsul (CapsNets).

Penelitian selanjutnya Li (2020) dengan judul "Celeb-DF: *A Large-scale Challenging Dataset for Deepfake Forensics*" menjelaskan tentang deteksi *deepfake* menggunakan perbandingan beberapa model pembelajaran mesin seperti *XceptionNet* dan DSP-FWA pada dataset Celeb-DF. Penelitian lain oleh De (2014) dengan judul "*Face liveness detection using dynamic texture*" menjelaskan tentang mendeteksi pemalsuan wajah menggunakan ekstensi *spatiotemporal* (tekstur dinamis) dari operator *local binary pattern*. Dengan pendekatan untuk mempelajari dan mendeteksi struktur dan dinamika tekstur mikro wajah yang menjadi ciri wajah asli.

Penelitian oleh Lionnie (2017) dengan judul "*A study on pre-processing methods for copy-move forgery detection based on SIFT*" menjelaskan tentang deteksi gambar palsu copy-move menggunakan metode pengolahan citra digital pemrosesan namun tanpa menggunakan model berbasis pembelajaran mesin. Peneltian lain oleh Dave (2020) dengan judul "*Spoof Detection Using Local Binary Pattern In Face*" menjelaskan tentang mendeteksi wajah palsu dan wajah asli, menggunakan *local binary pattern*.

Tabel 2.1 : Literatur Sejenis

Judul Peneltian	Penulis	Dataset	Gaussian	LBP	MTCNN	Model Klasifikasi
FaceForensics++: Learning to Detect Manipulated Facial Images	(Rossler et al., 2019)	FaceForensics++	–	–	–	*XceptionNet*
Face Anti-Spoofing Based On Color Texture Analysis	(Boulkenafet et al., 2015)	CASIA face anti-spoofing and ReplayAttack	–	✓	–	Support Vector Machine (SVM)
Detecting *Deepfakes* with Metric Learning	(Kumar et al., 2020)	FF++, Celeb-DF	–	–	✓	*XceptionNet*, Triplet Network
Face anti-spoofing with multifeature videolet aggregation	(Siddiqui et al., 2016)	CASIA-FASD, MSU mobile face spoofing database, 3D-MAD	–	✓	–	Support Vector Machine (SVM)
On the Detection of Digital Face Manipulation	(Dang et al., 2020)	DFFD	–	–	–	*XceptionNet*
iCaps-Dfake: An Integrated Capsule-Based Model for *Deepfake* Image and Video Detection	(Khalil et al., 2021)	DFDC-P	–	✓	–	*A capsule network* (CapsNet)
Celeb-DF: A Large-scale Challenging Dataset for *Deepfake* Forensics	(Li et al., 2020)	Celeb-DF(V2)	–	–	–	*XceptionNet*, DSP-FWA

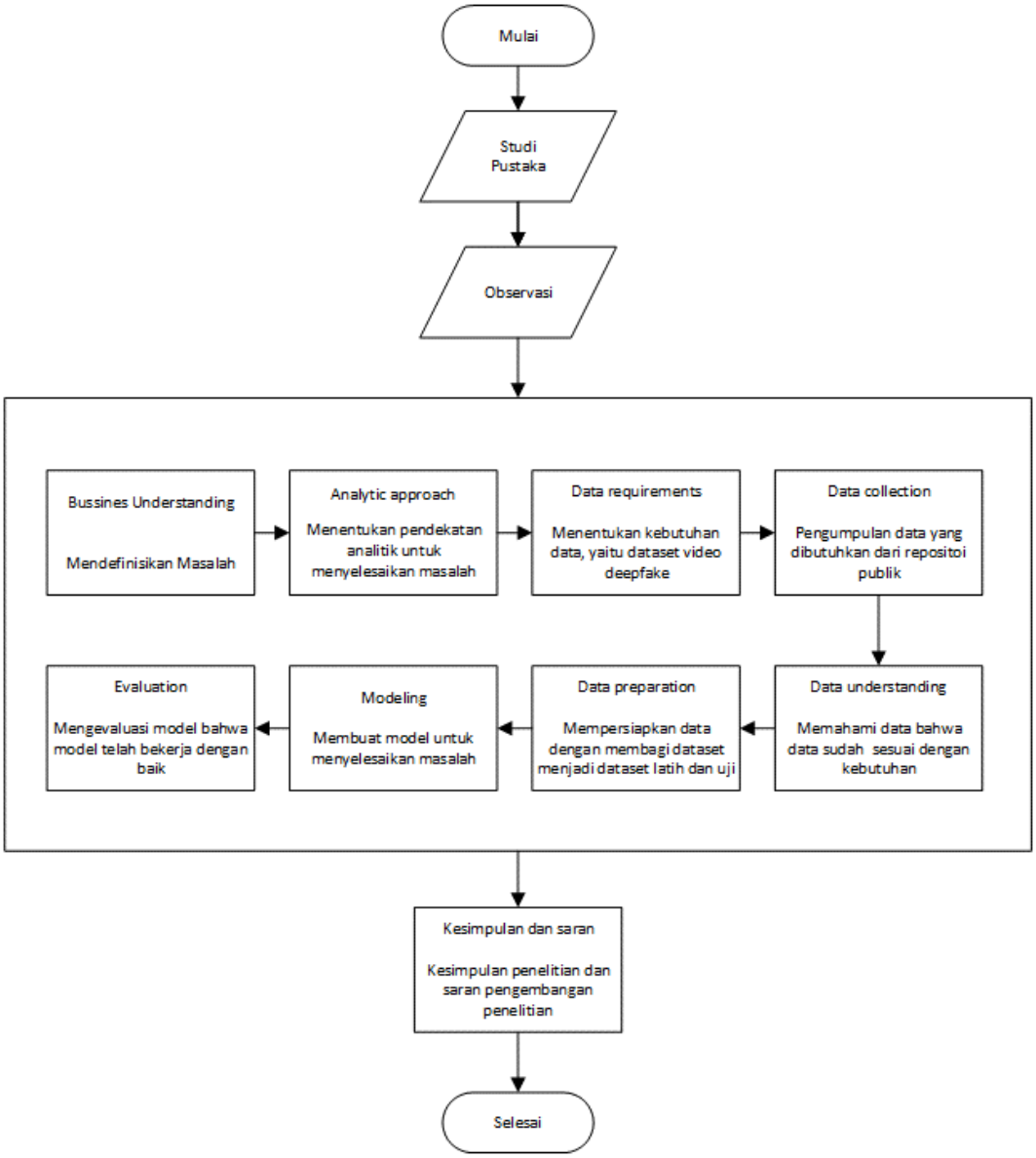
Face liveness detection using dynamic texture	(De et al., 2014)	Replay-Attack Database, CASIA Face Anti-Spoofing Database	–	✓	–	*linear discriminant analysis* (LDA)
A study on pre-processing methods for copy-move forgery detection based on SIFT	(Lionnie et al., 2017)	MICC F220	✓	–	–	SIFT
Spoof Detection Using Local Binary Pattern In Face	(Dave, 2020)	NUAA , CASIA	–	✓	–	*Support Vector Machine* (SVM)

Penulis melakukan penelitian mengenai klasifikasi video *deepfake* dengan menggunakan dataset sebesar 1000 data tiap kelas sehingga total data yang penulis gunakan sebesar 2000 data dari keseluruhan dataset Celeb-DF. Penulis hanya menggunakan dataset yang kecil berdasarkan penelitian oleh Francois Chollet (2016) yang menunujukkan bahwa dapat membuat model klasifikasi yang memiliki performa cukup baik meskipun hanya menggunakan dataset yang sedikit.

. Dengan beberapa aspek penelitian terkait sebelumnya metode yang diusulkan berupa melakukan pengambilan data wajah pada *frame* video menggunakan MTCNN (dengan jeda 15 detik dalam pengambilan *frame* pada proses pembuatan dataset), kemudian menerapkan *filter gaussian* pada data wajah tersebut untuk mengurangi adanya kebisingan pada gambar, selanjutnya dilakukan proses *Local Binary Pattern* untuk mendapatkan tekstur gambar, kemudian dilakukan klasifikasi data wajah tersebut menggunakan model *XceptionNet* dan model *ResNet*-50 sebagai pembanding karena berdasarkan penelitian oleh Li & Lyu (2018) menunjukkan hasil bahwa model *ResNet*-50 memiliki hasil terbaik dibandingkan dengan VGG16, *ResNet*101 and *ResNet*152 dalam mendeteksi *deepfake*.

BAB III
METODE PENELITIAN

3.1 Kerangka Berpikir



Gambar 3.1 : Kerangka Berpikir

3.2 Metode Pengumpulan Data

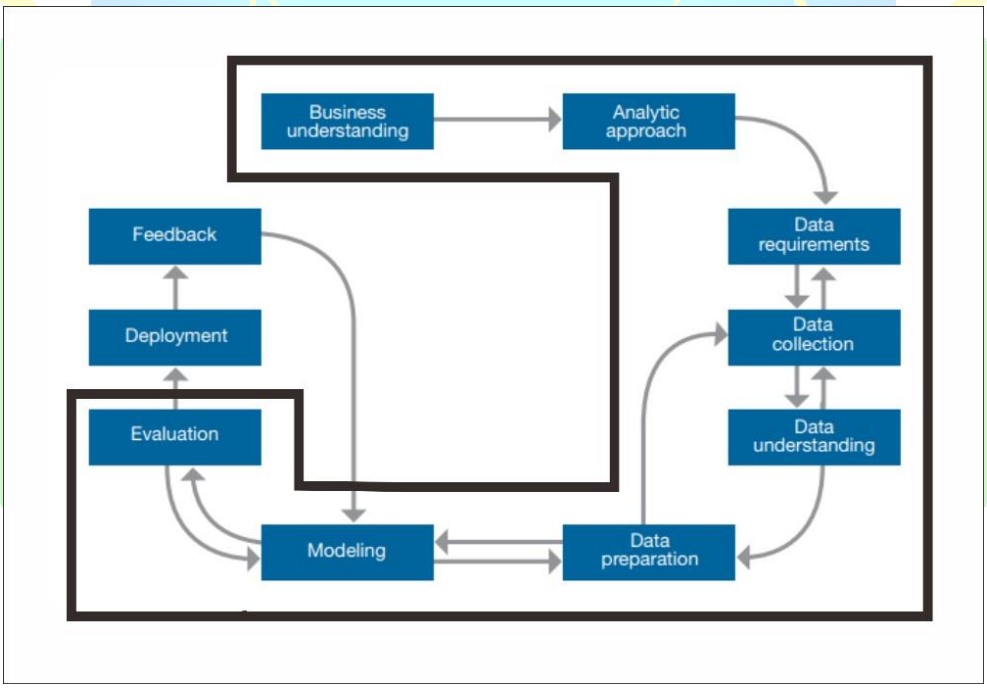
3.2.1 Studi Pustaka

Penulis melakukan studi pustaka dengan mengumpulka data pustaka berupa beberapa jurnal, buku, dan situs online yang digunakan sebagai sumber data dan informasi. Sumber data dan informasi tersebut diperlukan penulis untuk membantu penyelesaian bab pendahuluan, landasan teori, dan metode penelitian

3.2.2 Observasi

Penulis menggunakan observasi dengan mengambil data berupa dataset video dari internet.

3.3 Metode Implementasi Eksperimen



Gambar 3.2 : Gambar Diagram Data science methodology yang digunakan

Dalam hal ini peneliti hanya menggunakan delapan dari sepuluh langkah yang ada yaitu tahap business understanding hingga evaluation, untuk tahap

deployment dan feedback tidak diperlukan dalam penelitian ini, karena penelitian ini hanya sebatas pengujian.

Peneliti menggunakan alur penelitian dengan tahap sebagai berikut :

1. **Business understanding**

   Pada tahap ini penulis mendefinisikan masalah penelitian yaitu, karena kemudahan dalam pembuatan dan semakin banyaknya video *deepfake*. Diperlukan model untuk mengklasifikasi apakah sebuah video merupakan video *deepfake* atau video asli.

2. **Analytic approach**

   Pada tahap ini penulis menentukan pendekatan analitik berupa model prediksi yaitu, metode dengan model klasifikasi menggunakan model *XceptionNet* dan *ResNet*-50 sebagai pembanding, menggunakan pemrosesan gambar filter gaussian dan LBP, dan menggunakan MTCNN untuk mengekstraksi wajah dari *frame* video.

3. **Data requirements**

   Penulis menentukan kebutuhan dataset berupa data dengan jumlah 1000 gambar wajah asli dan 1000 gambar wajah *deepfake* dengan format ".jpg".

4. **Data collection**

   Penulis mengumpulkan data dengan jumlah 1000 gambar wajah asli dan 1000 gambar wajah *deepfake* dari dataset Celeb-DF(V2) sebagai dasar dataset.

5. **Data understanding**

   Penulis memahami isi dan memastikan data yang dikumpulkan sudah sesuai kebutuhan yaitu, data berupa gambar wajah asli dan *deepfake* berformat ".jpg".

6. **Data preparation**

   Persiapan data dilakukan dengan melakukan proses pembagian dataset menjadi data latih dan data uji.

7. **Modeling**

   Penulis membuat model berupa model prediktif untuk mengklasifikasi gambar apakah merupakan video *deepfake* atau asli, untuk kemudian dilatih dan diuji menggunakan dataset.

8. **Evaluation**

   Penulis mengevaluasi hasil dari performa model untuk memastikan model yang dibuat sudah sesuai dengan tujuan penelitian.

BAB IV
IMPLEMENTASI

4.1 Business Understanding dan Pendekatan Analisis

Penulis melakukan identifikasi masalah yang berdasarkan latar belakang penelitian, yaitu karena semakin banyaknya video *deepfake* dan kemudahan dalam pembuatannya, diperlukan model untuk mengklasifikasi video, apakah merupakan video *deepfake* atau video asli, dalam penelitian ini video berformat ".mp4".

4.2 Analytic Approach

Pendekatan analitik yang dilakukan adalah dengan membuat dan menguji sebuah model klasifikasi, yaitu metode yang menggunakan MTCNN untuk mengekstraksi wajah dari *frame* video, *filter gaussian* dan LBP untuk pemrosesan data gambar dan menggunakan model *XceptionNet* dan *ResNet*-50 untuk klasifikasi, yaitu memprediksi jawaban seperti "asli" atau "*deepfake*" dari sebuah video, penulis memilih model ini dengan pertimbangan hasil penelitian oleh Rossler (2019) untuk model *XceptionNet* dan penelitian oleh Rosebrock (2019) untuk model *ResNet*-50.

4.3 Data Requirement

Pada tahap ini, penulis menentukan kebutuhan dataset yang akan digunakan dalam pembuatan model untuk klasifikasi video *deepfake*, yaitu :

- Dataset memiliki dua label, yaitu *real* dan *deepfake*.
- Dataset eksperimen merupakan gambar wajah berformat ".jpg", seperti pada umumnya dataset wajah palsu.
- Ukuran image 224x224 yang merupakan input default umumnya model deep learning.
- Dataset eksperimen memiliki jumlah 1000 data tiap kelasnya.
- Dataset sumber berupa video dengan durasi rata-rata 13 detik.

- Epoch yang digunakan sebesar 20 epoch berdasarkan contoh pada TensorFlow Core (2021).
- Dataset menggunakan 200 video dari sumber dataset Celeb-DF(V2), mengambil lima data wajah pada *frame* pervideo dengan jeda 15 detik dalam pengambilan *frame* untuk mendapatkan ekspresi atau posisi wajah yang berbeda, sehingga total data berjumlah 1000 data wajah perkelas.
- Proses pengambilan lima data wajah pada *frame* diambil secara urut pervideo sebagai dataset sedangkan keseluruhan *frame* dalam satu video berjumlah lebih kurang 300 *frame*.
- Menggunakan *learning rate* 0,0001 berdasarkan penelitian oleh Amin et al. (2019) karena dengan learning rate tersebut model memiliki performa terbaik.
- Dataset dasar merupakan video berwarna tanpa suara dan berformat ".mp4" yang merupakan format bawaan dari dataset Celeb-DF(V2).

4.4 Data Collection

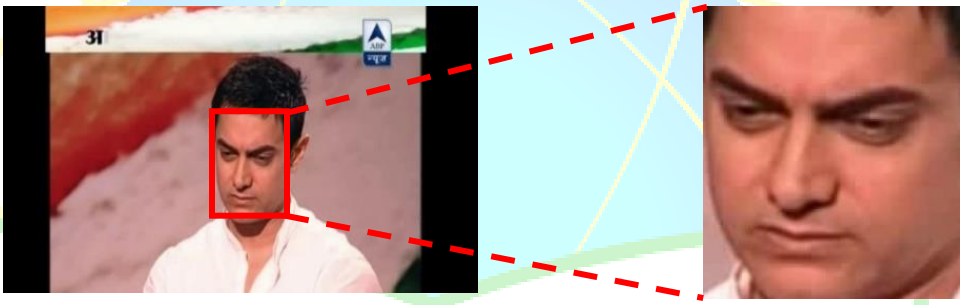
Penulis mengumpulkan data yang dibutuhkan untuk eksperimen. Penulis menggunakan dataset *deepfake* Celeb-DF (V2) sebagai dasar pembuatan dataset dalam penelitian ini. Menurut Li et al (2020) kualitas visual keseluruhan dari *Deepfake* yang disintesis video di Celeb-DF jauh lebih baik jika dibandingkan dengan kumpulan data yang ada, dengan artefak visual penting yang jauh lebih sedikit. Berdasarkan algoritma waktu rilis dan sintesis. Li et al (2019) mengkategorikan UADFV, DF-TIMIT, dan FF-DF sebagai generasi pertama dari dataset *Deepfake*, sedangkan DFD, DFDC, dan dataset Celeb-DF yang diusulkan adalah generasi kedua. Secara umum, dataset generasi kedua meningkat baik kuantitas maupun kualitasnya dibandingkan generasi pertama.

Tabel 4.1 : Tabel Keterangan Sumber Dataset

Nama	**Tipe File**	**Sumber**	**Jumlah Data**		**Tautan**
			Asli	***Deepfake***	
Celeb-DF V2	Video .mp4	Github	590	5639	https://github.com/yuezunli/celeb-*deepfake*forensics

4.4.1 Ekstraksi Wajah

Dalam pembuatan dataset, ekstraksi wajah dilakukan dengan menggunakan *Multi-task Cascaded Convolutional Networks* (MTCNN), dengan gambar wajah yang diambil dari *frame* dengan pengambilan *frame* memiliki jeda 15 detik untuk setiap *frame* dan total lima *frame* pervideo dari dataset sumber (Celeb-DF V2). Pengambilan *frame* dengan jeda 15 detik untuk setiap *frame* bermaksud untuk mendapatkan *frame* yang berisi ekspresi atau posisi wajah yang berbeda. Namun dalam penerapan model untuk prediksi, ekstraksi wajah dilakukan pada semua *frame* dari video yang diprediksi.



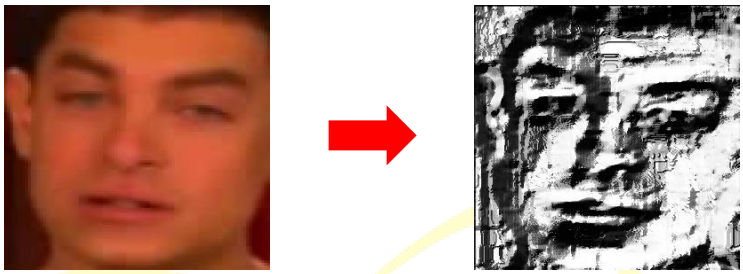
Gambar 4.1 : Contoh ekstraksi wajah dari *frame* menggunakan MTCNN

4.4.2 Proses Filter Gaussian dan Local Binary Pattern

Proses filter Gaussian dilakukan setelah proses ekstraksi wajah untuk mengurangi adanya kebisingan pada gambar dan selanjutnya dilakukan proses *Local Binary Pattern* untuk mendapatkan tekstur gambar. Proses ini dilakukan untuk sedikit mengupas hasil pengolahan citra dari pembuatan deepfake. Proses tersebut menggunakan kode berikut :

```
def preprocess(imag) :
    # settings for LBP
    radius = 2
    n_points = 8 * radius
    imgg = cv.GaussianBlur(imag,(5,5),2)
    _img = imgg
    for ichannel in range(_img.shape[2]):
       _img[:,:,ichannel] = lbp_(imgg[:,:,ichannel],
                                                n_points,radius)
```

```
    imggray = cv.cvtColor(_img, cv.COLOR_BGR2GRAY)
    return imggray
```



Gambar 4.2 : Contoh Sebelum dan Sesudah Penerapan Proses Filter Gaussian dan LBP

4.4.3 Dataset

Berdasarkan table 4.2 Dataset yang diperoleh berupa gambar wajah, memilki 2 kelas, yaitu *real* dan *deepfake*, dan memiliki jumlah 1000 data tiap kelasnya dan berformat ".jpg".

Tabel 4.2 : Keterangan Dataset

Kelas	Label	Jumlah Total Data	Format
deepfake	0	1000	".jpg"
real	1	1000	".jpg"

4.5 Data Understanding

Pada tahap ini penulis memastikan data yang diperoleh telah memenuhi kebutuhan yang telah ditetapkan, yaitu : data berupa gambar wajah, memilki 2 kelas, yaitu *real* dan *deepfake*, dan memiliki jumlah 1000 data tiap kelasnya dan berformat ".jpg".

4.6 Data Preparation

Menurut Madiraju (2020) data harus dibagi sedemikian rupa sehingga tidak ada yang terlalu tinggi, yang lebih bergantung pada jumlah data yang dimiliki. Jika data terlalu kecil maka tidak ada pemisahan yang akan memberi varian yang

memuaskan sehingga harus melakukan validasi silang tetapi jika data sangat besar maka tidak masalah apakah memilih pemisahan 80:20 atau pemisahan 90:10.

Dalam penelitian ini penulis membagi dataset menjadi dataset latih dan dataset validasi dengan perbandingan 80% dataset latih dan 20% dataset validasi. Penulis menggunakan library scikit-learn untuk melakukan proses pembagian dataset menggunakan kode sebagai berikut :

```
trainreal_dir, valreal_dir = train_test_split(os.listdir
                                (real_dir), test_size = 0.2)
trainfake_dir, valfake_dir = train_test_split(os.listdir
                                (fake_dir), test_size = 0.2)
```

Dari proses pembagian dataset didapatkan hasil sebagai berikut :

Tabel 4.3 : Tabel Pembagian Dataset

Kelas	**Label**	**Jumlah Total Data**	**Data Latih (80%)**	**Data Validasi (20%)**	**Format**
deepfake	0	1000	800	200	".jpg"
real	1	1000	800	200	".jpg"

4.7 Modeling

Dalam tahap ini, penulis membuat model untuk klasifikasi video *deepfake*. Sesuai dengan tujuan penelitian, penulis menggunakan pemodelan prediktif yang merupakan proses yang menggunakan data dan probabilitas untuk memprediksi hasil, yaitu model prediktif digunakan untuk menentukan apakah sebuah video merupakan video asli atau video *deepfake*.

Model yang diunduh memiliki beberapa parameter, yaitu ukuran input (*input_shape*), bobot model (*weights*), dan menggunakan lapisan atas atau tidak(*include_top)*.

a. *XceptionNet*

```
base_model = Xception(
    weights="imagenet",
```

```
    input_shape=(img_height,img_width,3),
    include_top=False)
```

b. *ResNet*-50

```
base_modelr = tf.keras.applications.ResNet50(
    weights="imagenet",
    input_shape=(img_height,img_width,3),
    include_top=False)
```

Kemudian semua lapisan model yang telah diunduh ditetapkan tidak dilatih atau dibekukan, agar disaat proses pelatihan bobot bawaan model tidak berubah.

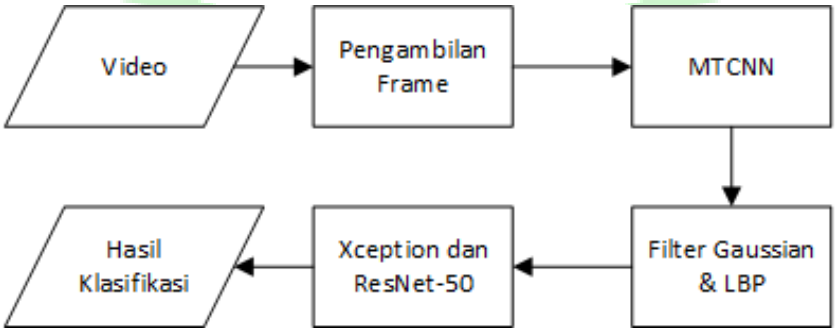
```
base_model.trainable = False
```

.

Model yang telah diunduh dan dibekukan kemudian digunakan untuk membuat model baru, seperti ditunjukkan pada kode dibawah ini.

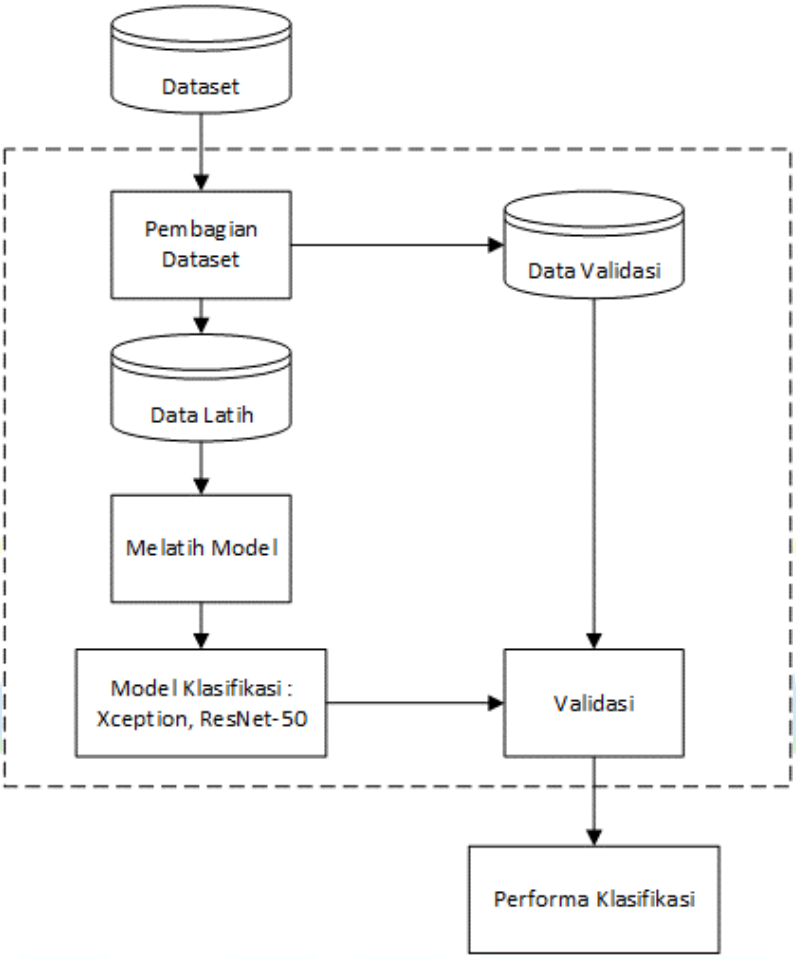
```
model = Sequential()
model.add(base_model)
model.add(AveragePooling2D(pool_size=(7,7)))
model.add(Flatten())
model.add(Dense(2, activation='softmax'))
```

Metode yang diusulkan dibuat dengan alur video sebagai input, kemudian proses pengambilan wajah dari *frame* menggunakan MTCNN. Kemudian data wajah tersebut diproses menggunakan filter Gaussian selanjutnya untuk mendapatkan tekstur gambar menggunakan LBP. Hasil dari proses sebelumnya, selanjutnya sebagai input model *XceptionNet* dalam klasifikasi video *deepfake* dan model *ResNet*-50 sebagai pembanding.



Gambar 4.3 : Alur Metode

4.7.1 Pelatihan (*Training*)



Gambar 4.4 : Alur Skenario Proses Eksperimen

Gambar 4.4 merupakan alur scenario proses eksperimen penelitian ini. Tahap pertama mengumpulkan dataset. Kemudian, membagi data menjadi data pelatihan dan validasi. Hasil performa model yang digunakan adalah AUC, akurasi, *loss*, dan f1- *score*, dan *confusion matrix*. Untuk melatih model diperlukan untuk menskalakan piksel input antara -1 dan 1 pada data yang merupakan skala input default model.

```
train_dsgen = ImageDataGenerator(
             preprocessing_function=pix,
                          horizontal_flip = True,
                          vertical_flip = True,
                          shear_range = 0.2)
```

```
val_dsgen = ImageDataGenerator(
              preprocessing_function=pix)
```

Setelah dilakukan pelabelan dataset, data diubah resolusinya menjadi 224x224 piksel menyesuaikan input model yang digunakan menggunakan *flow_from_directory* dari Keras.

```
batch_size = 100
img_height = 224
img_width = 224
train_ds = train_dsgen.flow_from_directory(
        train_dir,
        target_size=(img_height, img_width),
        batch_size=batch_size,
        class_mode='categorical')
val_ds = val_dsgen.flow_from_directory(
        validation_dir,
        target_size=(img_height, img_width),
        batch_size=batch_size,
        class_mode='categorical',
        shuffle=False)
```

Selanjutnya untuk mendapatkan daftar label dan kelas menggunakan kode berikut :

```
class_names = train_ds.class_indices
print(class_names)
# Obtain the list of the classes
label = list(class_names.keys())
print("The list of classes: ", label)
```

```
{'deepfake': 0, 'real': 1}
The list of classes:  ['deepfake', 'real']
```

Gambar 4.5 : Label dan Kelas Data

Dataset yang telah dikumpulkan kemudian digunakan untuk melatih model yang dibuat. Dalam melatih model penulis menggunakan 20 epoch berdasarkan contoh pada TensorFlow Core (2021), dan menggunakan categorical crossentropy untuk menghitung nilai *loss* yang didasari penelitian

oleh Ayala-Acevedo (2017) dan menggunakan optimizer Adam dengan learning rate 0,0001 berdasarkan penelitian oleh Amin et al. (2019).

```
METRICS = [
      tf.keras.metrics.CategoricalAccuracy(name='accuracy'),
      tf.keras.metrics.AUC(name='auc', curve='ROC')]
```

```
cross_entropy = tf.keras.losses.CategoricalCrossentropy(from_logi
ts=True)
optimizer = tf.keras.optimizers.Adam(learning_rate=0.0001)
model.compile(optimizer= optimizer,
    loss = cross_entropy,
    metrics = [METRICS])
```

```
start_time = time.time()
history= model.fit(
    train_ds,
    steps_per_epoch= step_train,
    epochs=20,
    validation_data=val_ds,
    validation_steps= step_val,
    verbose=1)

final_accuracy = history.history["val_accuracy"][-5:]
print("FINAL ACCURACY MEAN-5: ", np.mean(final_accuracy))
print("TRAINING TIME: ", time.time() - start_time, " sec")
```

Setelah proses melatih model selesai, hasil performa model pada set pelatihan dan set validasi, selanjutnya ditampilkan menggunakan grafik dengan *library* matplotlib. Proses naik turunnya performa model pada akurasi, *loss*, dan AUC dapat diketahui melalui garfik yang digunakan pada set data latih maupun set data validasi.

```
# list all data in history
print(history.history.keys())
acc = history.history['accuracy']
epochs = range(len(acc))
# summarize history for accuracy
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Training and validation accuracy')
plt.ylabel('accuracy')
plt.xlabel(epochs)
plt.legend(['Training Accuracy', 'Validation Accuracy'],
                                          loc='upper left')
plt.show()
# summarize history for loss
```

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Training and validation loss')
plt.ylabel('loss')
plt.xlabel(epochs)
plt.legend(['Training Loss', 'Validation Loss'],loc='upper left')
plt.show()
# summarize history for AUC
plt.plot(history.history['auc'])
plt.plot(history.history['val_auc'])
plt.title('Training and validation AUC')
plt.ylabel('auc')
plt.xlabel(epochs)
plt.legend(['Training AUC', 'Validation AUC'],loc='upper left')
plt.show()
```

Menurut Brownlee (2018) jika sedang mengerjakan masalah klasifikasi, skor terbaik adalah akurasi 100%, jadi semakin mendekati 100% maka akurasi semakin baik. Grandperrin (2021) mengatakan sebagai manusia, cara paling alami untuk menafsirkan prediksi sebagai "ya" diberikan skor kepercayaan antara 0 dan 1 adalah dengan memeriksa apakah nilainya di atas 0,5 atau tidak. 0,5 ini adalah nilai ambang batas, dengan kata lain, 0,5 adalah skor kepercayaan minimum di atas yang dianggap prediksi sebagai "ya". Jika di bawah, maka dianggap prediksi sebagai "tidak".

Loss adalah indikasi untuk prediksi yang buruk. Artinya, *loss* (kerugian) adalah angka yang menunjukkan seberapa buruk prediksi model. Jika prediksi model sempurna, kerugiannya adalah nol, jika tidak, kerugiannya lebih besar. Tujuan dari pelatihan model adalah untuk menemukan satu set bobot dan bias yang memiliki kerugian rendah (Google 2020).Kemudian dilakukan evaluasi performa klasifikasi model pada setiap kelas set data validasi menggunakan *confusion matrix* dengan *library* scikitlearn.

Dengan menggunakan *confusion matrix* dapat diketahui berapa jumlah prediksi model yang benar dan yang salah pada setiap kelas, dan dapat diketahui juga berapa prediksi model yang salah ataupun benar dari total set

data validasi. Penulis juga menggunakan *classification_report* dari scikit-learn untuk mendapatkan laporan klasifikasi dari model yang dibuat.

```
val_ds.reset()
y_pred = model.predict(val_ds, step_val)
y_pred = np.argmax(y_pred, axis=1)
y_test = val_ds.classes
cm = confusion_matrix(y_test, y_pred)
labels = label
plt.figure()
plot_confusion_matrix(cm=cm, classes=label,
                    title='Non Normalized confusion_matrix')
from sklearn.metrics import classification_report
print (classification_report(y_test, y_pred))
```

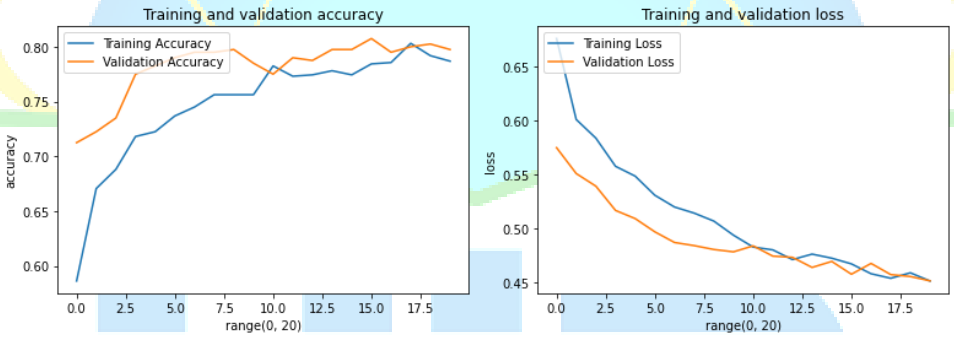
BAB V
HASIL DAN PEMBAHASAN

Bab ini membahas hasil dari penelitian ini, penulis menggunakan model. Performa model yang dibuat dapat dilihat dari hasil uji yang dilakukan.

5.1 Evaluasi

Pada tahap ini, penulis melakukan evaluasi dari hasil tahapan proses yang telah dilakukan sebelumnya. Evaluasi dilakukan dengan tujuan untuk memastikan hasil yang didapat sesuai dengan tujuan penelitian. Setelah model dilatih dan divalidasi didapatkan hasil berupa skor AUC, *Confusion Matrix*, dan F1-*Score*.

5.1.1 Hasil Pelatihan Model *XceptionNet*



Gambar 5.1 : Grafik Hasil Pelatihan Model *XceptionNet* dengan warna oranye merupakan hasil pada set validasi dan biru pada set pelatihan. Grafik akurasi model berada pada bagian kiri, dan *loss* model pada bagian kanan.

Berdasarkan tabel 5.1 dapat diketahui model *XceptionNet* memiliki hasil pelatihan dengan *learning rate* 0,0001 dan *epoch* 20, berupa nilai akurasi 0.79, dan *loss* 0.45 pada set validasi dan nilai akurasi 0.78, dan *loss* 0.45 pada set pelatihan.

Tabel 5.1 : Tabel Hasil Pelatihan Model *XceptionNet*

Model *XceptionNet*	**Set Pelatihan**	**Set Validasi**
Akurasi	0.78	0.79
Loss	0.45	0.45

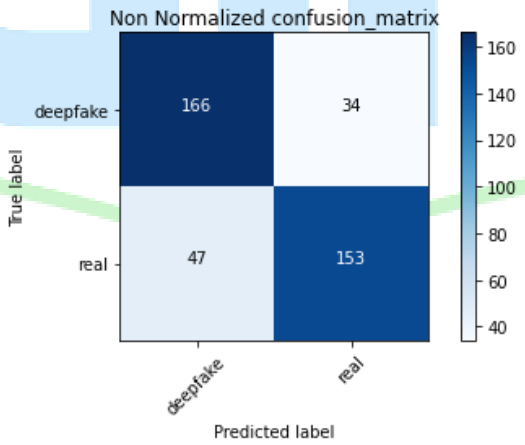
Penulis menggunakan *Classification report* dari *library* sklearn, untuk mendapatkan *precision*, *recall*, serta *f1-score* dari performa klasifikasi model pada dataset validasi.

	precision	recall	f1-score	support
0	0.78	0.83	0.80	200
1	0.82	0.77	0.79	200
accuracy			0.80	400
macro avg	0.80	0.80	0.80	400
weighted avg	0.80	0.80	0.80	400

Gambar 5.2 Classification Report Model *XceptionNet*

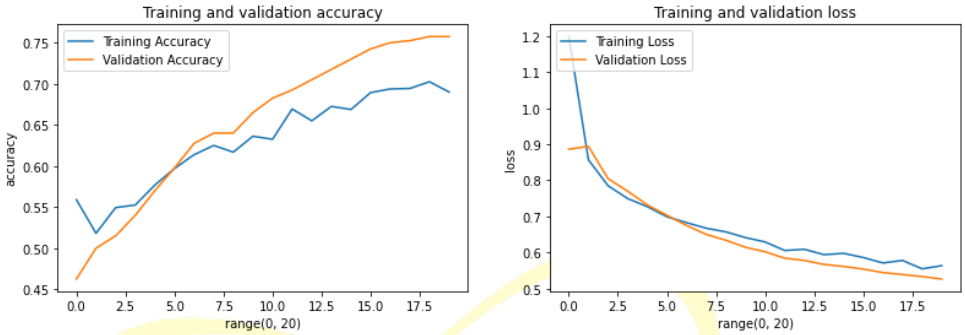
Berdasarkan gambar 5.2, model *XceptionNet* memiliki performa klasifikasi yang cukup baik, dengan nilai precision untuk 2 kelas berada diatas 70%, dan nilai precision tertinggi dimiliki oleh kelas *real* dengan label 1 sebesar 82% dan kelas *deepfake* dengan label 0 sebesar 78%. Selain itu metrik recall dan f1-score berada diatas 70%.

Penulis juga menggunakan *Confusion matrix* untuk menunjukkan performa klasifikasi benar dan salah dari setiap label data oleh model.. Dari gambar 5.3, dapat diketahui bahwa model *XceptionNet* memiliki kesalahan prediksi 34 kesalahan dari 200 data pada kelas *deepfake*, dan 47 kesalahan dari 200 data pada kelas *real*.



Gambar 5.3 : *Confusion Matrix* Model *XceptionNet*

5.1.2 Hasil Pelatihan Model *ResNet*-50



Gambar 5.4 : Grafik Hasil Pelatihan Model *ResNet*-50 dengan warna oranye merupakan hasil pada set validasi dan biru pada set pelatihan. Grafik akurasi model berada pada bagian kiri, dan *loss* model pada bagian kanan.

Berdasarkan tabel 5.2 dapat diketahui model *ResNet*-50 memiliki performa hasil pelatihan dengan *learning rate* 0,0001 dan *epoch* 20, berupa nilai akurasi 0.75, dan *loss* 0.52 pada set validasi dan nilai akurasi 0.69, dan *loss* 0.56 pada set pelatihan.

Tabel 5.2 : Tabel Hasil Pelatihan Model *ResNet*-50

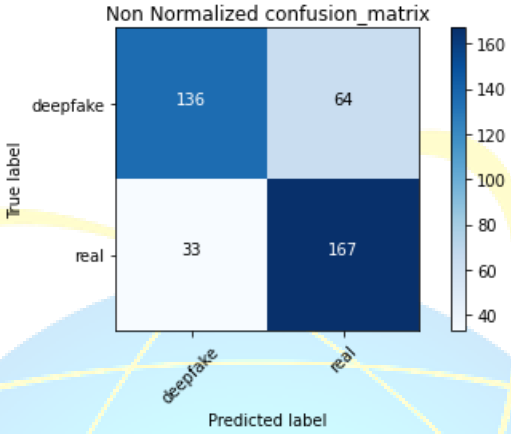
Model *ResNet-50*	**Set Pelatihan**	**Set Validasi**
Akurasi	0.69	0.75
Loss	0.56	0.52

Dari gambar 5.5, terlihat bahwa model *ResNet-50* memiliki performa klasifikasi yang cukup baik, ini terlihat dengan nilai *precision* untuk 2 kelas berada diatas 70%, dengan nilai *precision* tertinggi dimiliki oleh kelas *deepfake* dengan label 0 sebesar 80% dan kelas *real* dengan label 1 sebesar 0.72. Dan metrik *recall* dan f1-*score* memiliki nilai diatas 60%.

	precision	recall	f1-score	support
0	0.80	0.68	0.74	200
1	0.72	0.83	0.77	200
accuracy			0.76	400
macro avg	0.76	0.76	0.76	400
weighted avg	0.76	0.76	0.76	400

Gambar 5.5 : *Classification Report* Model *ResNet*-50

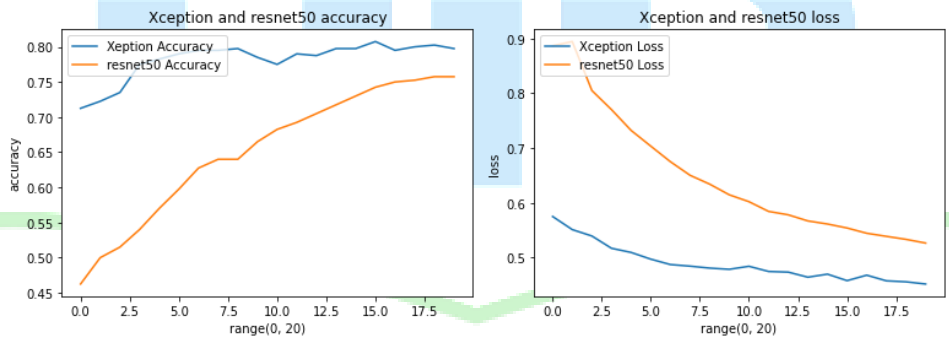
Berdasarkan gambar 5.6, dapat diketahui model *ResNet*-50 memiliki kesalahan prediksi 64 kesalahan dari 200 data pada kelas *deepfake*, dan 33 kesalahan dari 200 data pada kelas *real*.



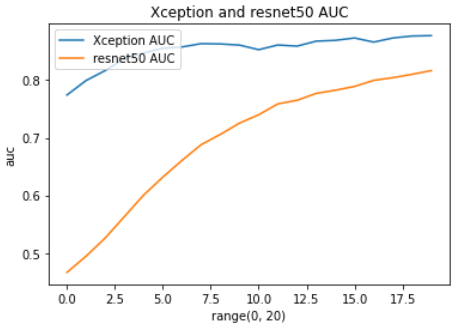
Gambar 5.6 : *Confusion Matrix* Model *ResNet*-50

5.1.3 Bagaimana performa metode *Filter Gaussian* dan LBP dengan model *XceptionNet* dan model *ResNet*-50 sebagai pembanding dalam klasifikasi video *deepfake*?

Dalam menjawab pertanyaan ini, penulis membuat perbandingan performa AUC, akurasi, *loss*, *F1-Score* dan *Confusion Matrix* pada set validasi.



Gambar 5.7 : Grafik perbandingan hasil pelatihan pada set validasi. Warna oranye merupakan model *ResNet*-50 dan biru merupakan model *XceptionNet*. Grafik perbandingan akurasi model berada pada bagian kiri, dan perbandingan *loss* model pada bagian kanan.



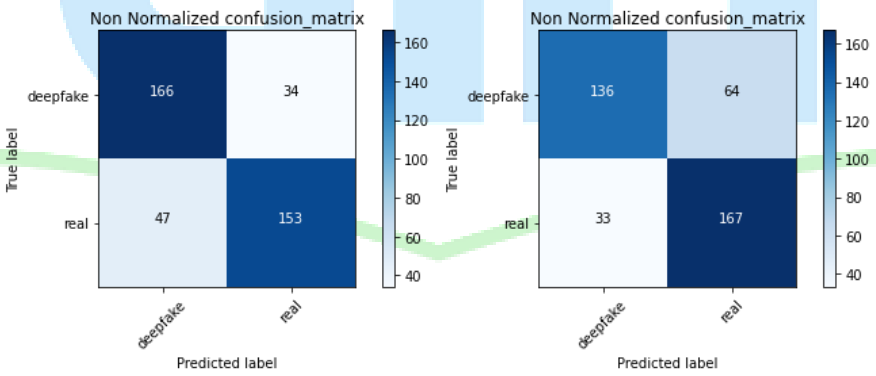
Gambar 5.8 : Grafik perbandingan performa nilai AUC model. Warna oranye merupakan model *ResNet*-50 dan biru merupakan model *XceptionNet*.

Penulis juga membuat perbandingan hasil performa model dengan membandingkan nilai AUC, akurasi, dan *loss* antara model *XceptionNet* dan model *ResNet*-50.

Tabel 5.3 : Tabel Perbandingan Performa Model Pada Set Validasi

Model Klasifikasi	**AUC**	**Akurasi**	***Loss***
XceptionNet	**0.87**	**0.79**	**0.45**
ResNet-50	0.81	0.75	0.52

Berdasarkan Tabel 5.3, model *XceptionNet* memiliki kinerja berupa AUC, akurasi dan *loss* lebih baik dari model *ResNet*-50 dengan nilai AUC sebesar 0.87, akurasi 0.79, dan *loss* 0.45. Selain itu penulis melakukan perbandingan pada Confusion Matrix dari kedua model seperti ditunjukkan pada gambar 5.9.



Gambar 5.9 : Perbandingan performa model pada *Confusion Matrix*. *Confusion matrix* model *XceptionNet* berada pada bagian kiri, dan *Confusion matrix* model *ResNet*-50 pada bagian kanan.

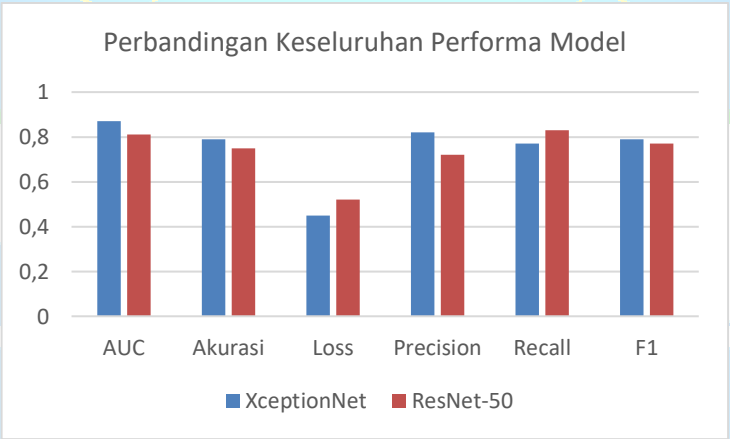
Dari gambar 5.9 diperolah perincian hasil perbandingan performa model sebagai berikut :

Tabel 5.4 : Tabel Perbandingan Confusion Matrix

Model	**TP**	**FP**	**FN**	**TN**
XceptionNet	**166**	47	**34**	153
ResNet-50	136	**33**	64	**167**

Tabel 5.5 : Tabel Perbandingan Keseluruhan Performa Model

Model	**AUC**	**Akurasi**	***Loss***	**Precision**	**Recall**	**F1**
XceptionNet	**0.87**	**0.79**	**0.45**	**0.82**	0.77	**0.79**
ResNet-50	0.81	0.75	0.52	0.72	**0.83**	0.77

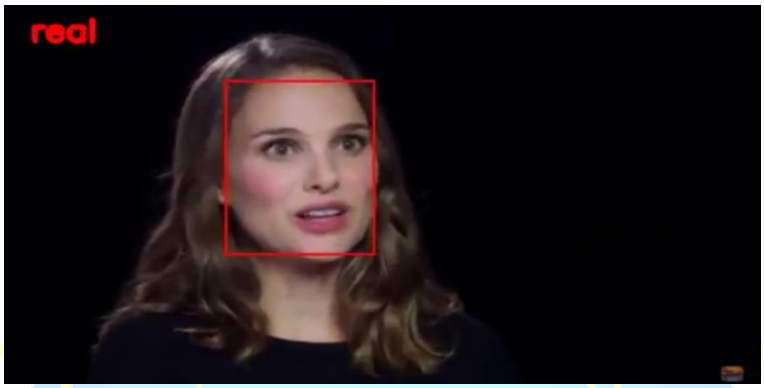


Gambar 5.10 : Grafik Perbandingan Keseluruhan Performa Model

Berdasarkan hasil dari perbandingan nilai AUC, akurasi, *loss*, *Confusion Matrix*, dan F1-*score* secara umum model *XceptionNet* memiliki performa lebih tinggi dibandingkan dengan performa model *ResNet*-50.

5.1.4 Pengujian

Hasil video yang telah diklasifikasi akan ada kotak pembatas pada wajah yang diklasifikasi dalam frame, dan hasil klasifikasi akan muncul pada sisi kiri atas pada frame.



Gambar 5.11 : Gambar Hasil Klasifikasi Real Pada Frame

Gambar 5.12 : Gambar Hasil Klasifikasi Deepfake Pada Frame

Penulis melakukan pengujian performa klasifikasi setiap model dalam skala 10 video untuk mengetahui berapa video yang diklasifikasikan dengan benar oleh model dari 10 video, dengan kelas 10 video *real* dan 10 video *deepfake* dari dataset Celeb-DF diluar dataset penelitian, sehingga dapat diketahui hasil klasifikasi video apakah sesuai dengan kelas aslinya. Pada hasil pengujian akan ditunjukkan nama video, total frame video, waktu klasifikasi, dan hasil rata-rata dari prediksi seluruh frame pada video.

Dari hasil pengujian klasifikasi pada 10 video real menunujukkan hasil 9 dari 10 video real diklasifikasikan sebagai video *real* dan sisanya 1 video diklasifikasikan sebagai video *deepfake* seperti ditunjukkan pada table 5.6.

Tabel 5.6 : Tabel Hasil Klasifikasi Model XceptionNet Pada Video Real

Model XceptionNet				
Video Real				
No.	Nama	Total Frame	Waktu	Hasil Rata2
1.	id41_0008.mp4	329	551.92 detik	**real_71.77%_**
2.	id51_0004.mp4	447	573.10 detik	**real_79.26%_**
3.	id38_0006.mp4	467	451.20 detik	**real_73.64%_**
4.	id46_0004.mp4	310	283.98 detik	deepfake_66.46%_
5.	id41_0004.mp4	314	293.26 detik	**real_76.16%_**
6.	id45_0004.mp4	311	288.61 detik	**real_79.24%_**
7.	id37_0004.mp4	359	338.81 detik	**real_79.38%_**
8.	id51_0006.mp4	461	439.44 detik	**real_74.53%_**
9.	id36_0009.mp4	307	282.08 detik	**real_87.08%_**
10.	id51_0009.mp4	512	510.94 detik	**real_67.09%_**

Berdasarkan table 5.7 hasil pengujian klasifikasi model Xception pada 10 video deepfake memilki hasil 2 dari 10 video deepfake diklasifikasikan sebagai video *deepfake*, sedangkan sisanya 8 video diklasifikasikan sebagai video *real*.

Tabel 5.7 : Tabel Hasil Klasifikasi Model XceptionNet Pada Video Deepfake

Model XceptionNet				
Video Deepfake				
No.	Nama	Total Frame	Waktu	Hasil Rata2
1.	id21_id25_0008.mp4	322	279.20 detik	real_65.12%_
2.	id53_id50_0005.mp4	466	427.65 detik	real_62.77%_
3.	id31_id26_0006.mp4	365	337.31 detik	real_68.20%_
4.	id53_id56_0009.mp4	451	401.52 detik	real_72.31%_
5.	id37_id4_0007.mp4	473	418.73 detik	real_69.48%_
6.	id3_id37_0007.mp4	361	314.43 detik	real_81.53%_
7.	id0_id2_0008.mp4	464	422.09 detik	**deepfake_69.11%_**
8.	id41_id47_0008.mp4	329	296.81 detik	real_67.69%_
9.	id50_id55_0007.mp4	437	393.04 detik	real_66.18%_
10.	id1_id30_0004.mp4	409	324.34 detik	**deepfake_63.95%_**

Dari table 5.8 dapat diketahui model ResNet-50 mengklasifikasikan dengan benar 7 video sebagai video real dari 10 video real, dan 3 video diklasifikasikan sebagai video deepfake.

Tabel 5.8 : Tabel Hasil Klasifikasi Model ResNet-50 Pada Video Real

Model ResNet-50				
Video Real				
No.	Nama	Total Frame	Waktu	Hasil Rata2
1.	id41_0008.mp4	329	320.73 detik	**real_66.19%_**
2.	id51_0004.mp4	447	498.00 detik	**real_85.90%_**
3.	id38_0006.mp4	467	447.36 detik	**real_70.65%_**
4.	id46_0004.mp4	310	287.88 detik	deepfake_73.12%_
5.	id41_0004.mp4	314	300.63 detik	deepfake_65.88%_
6.	id45_0004.mp4	311	296.21 detik	**real_66.86%_**
7.	id37_0004.mp4	359	347.87 detik	**real_71.20%_**
8.	id51_0006.mp4	461	455.10 detik	**real_70.20%_**
9.	id36_0009.mp4	307	288.87 detik	**real_63.97%_**
10.	id51_0009.mp4	512	537.87 detik	deepfake_67.52%_

Berdasarkan tabel 5.9 model ResNet-50 berhasil mengklasifikasikan 6 video sebagai video deepfake dari 10 video deepfake dan sisanya 4 video diklasifikasikan sebagai video real

Tabel 5.9 : Tabel Hasil Klasifikasi Model ResNet-50 Pada Video Deepfake

Model ResNet-50				
Video Deepfake				
No.	Nama	Total Frame	Waktu	Hasil Rata2
1.	id21_id25_0008.mp4	322	738.72 detik	**deepfake_66.96%_**
2.	id53_id50_0005.mp4	466	642.36 detik	**deepfake_73.60%_**
3.	id31_id26_0006.mp4	365	357.47 detik	**deepfake_77.81%_**
4.	id53_id56_0009.mp4	451	432.90 detik	real_72.69%_
5.	id37_id4_0007.mp4	473	466.23 detik	real_65.68%_
6.	id3_id37_0007.mp4	361	344.74 detik	real_71.76%_
7.	id0_id2_0008.mp4	464	451.56 detik	**deepfake_86.23%_**
8.	id41_id47_0008.mp4	329	317.51 detik	**deepfake_68.12%_**
9.	id50_id55_0007.mp4	437	424.04 detik	real_67.32%_
10.	id1_id30_0004.mp4	409	350.73 detik	**deepfake_71.65%_**

Berdasarkan hasil pengujian klasifikasi model, model XceptionNet memiliki hasil klasifikasi lebih unggul dari model ResNet-50 pada video real, dan model ResNet-50 memiliki hasil klasifikasi lebih unggul dari model XceptionNet pada klasifikasi video deepfake dalam skala 10 video.

Tabel 5.10 : Tabel Perbandingan Hasil Pengujian Klasifikasi Model

	Model XceptionNet		Model ResNet-50	
	Video Real	Video Deepfake	Video Real	Video Deepfake
Klasifikasi Benar	**9**	2	7	**6**
Klasifikasi Salah	**1**	8	3	**4**

BAB VI
PENUTUP

6.1 Kesimpulan

Dalam penelitian ini penulis mengusulkan metode klasifikasi video *deepfake* menggunakan pemrosesan gambar *filter Gaussian* dan *Local Binary Pattern* dengan model klasifikasi menggunakan model *XceptionNet* dan *ResNet-50* sebagai pembanding. Hasil eksperimen menunjukkan bahwa metode yang diusulkan dengan menggunakan dataset sebesar 1000 data perkelas, memiliki nilai AUC sebesar 0.87 dan nilai Akurasi sebesar 0.79 , lebih baik jika dibandingkan dengan model ResNet-50.

6.2 Saran

Metode baru yang lebih kuat dalam menklasifikasi *deepfake* perlu dikembangkan untuk mengetahui apakah sebuah video merupakan *deepfake* atau asli, dimana teknologi pembuatan *deepfake* semakin banyak dan maju. Kualitas dataset yang mumpuni perlu diperbarui untuk meningkatkan kualitas dari model yang akan dibuat dengan mengikuti perkembangan pembuatan *deepfake*.

DAFTAR PUSTAKA

. I Made Suandi Putra MSc, CIA, C., Yullyan, SE, MAk, Ak., CPA, CIA, C., & Globaliia.org. (2017). PERSPEKTIF DAN PANDANGAN GLOBAL Kecerdasan Buatan (Artificial Intelligence/AI) - Pertimbangan untuk Profesi Audit Internal. *PERSPEKTIF DAN PANDANGAN GLOBAL Kecerdasan Buatan (Artificial Intelligence/AI)*, *9*(2), 1–9.

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2016). *TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems*. http://arxiv.org/abs/1603.04467

Afchar, D., Nozick, V., Yamagishi, J., & Echizen, I. (2019). MesoNet: A compact facial video forgery detection network. *10th IEEE International Workshop on Information Forensics and Security, WIFS 2018*. https://doi.org/10.1109/WIFS.2018.8630761

Aggarwal, V., & Kaur, G. (2018). A review:deep learning technique for image classification. *ACCENTS Transactions on Image Processing and Computer Vision*, *4*(11), 21–25. https://doi.org/10.19101/tipcv.2018.411003

Ahmad, A. (2017). *Mengenal Artificial Intelligence, Machine Learning, Neural Network, dan Deep Learning*. *October*.

Akbar1, S. N. A. F., Hendra2, & Supri Bin Hj. Amir3. (2020). *Perbandingan Kinerja Arsitektur Inception-V4 Dan Resnet-50 Dalam Mengklasifikasikan Citra Paru-Paru Terinfeksi Covid-19*. *2*.

Al-Sanjary, O. I., & Sulong, G. (2015). Detection of video forgery: A review of literature. *Journal of Theoretical and Applied Information Technology*, *74*(2), 207–220.

Almars, A. M. (2021). *Deepfakes Detection Techniques Using Deep Learning : A*

Survey. 20–35. https://doi.org/10.4236/jcc.2021.95003

Althnian, A., AlSaeed, D., Al-Baity, H., Samha, A., Dris, A. Bin, Alzakari, N., Abou Elwafa, A., & Kurdi, H. (2021). Impact of dataset size on classification performance: An empirical evaluation in the medical domain. *Applied Sciences (Switzerland)*, *11*(2), 1–18. https://doi.org/10.3390/app11020796

Amin, A., Sari, Y. A., & Adinugroho, S. (2019). Klon Perilaku Menggunakan Jaringan Saraf Tiruan Konvolusional Dalam Game SuperTuxKart. *Jurnal Pengembangan Teknologi Informasi Dan Ilmu Komputer (J-PTIIK) Universitas Brawijaya*, *3*(1), 866–875.

Ayala-Acevedo, A. (2017). *Image Classification: Fine-Tune CNN Model via Transfer Learning*. https://github.com/avn3r/image-classifier

Bayar, B., & Stamm, M. C. (2016). A deep learning approach to universal image manipulation detection using a new convolutional layer. *IH and MMSec 2016 - Proceedings of the 2016 ACM Information Hiding and Multimedia Security Workshop*, 5–10. https://doi.org/10.1145/2909827.2930786

Boulkenafet, Z., Komulainen, J., & Hadid, A. (2015). Face anti-spoofing based on color texture analysis. *Proceedings - International Conference on Image Processing, ICIP*, *2015-Decem*, 2636–2640. https://doi.org/10.1109/ICIP.2015.7351280

Brownlee, J. (2018). *How To Know if Your Machine Learning Model Has Good Performance*. Machine Learning Mastery. https://machinelearningmastery.com/how-to-know-if-your-machine-learning-model-has-good-performance/

J. Brownlee, "How to Perform Face Detection with Deep Learning," Deep Learning for Computer Vision, 2019. https://machinelearningmastery.com/how-to-perform-face-detection-with-classical-and-deep-learning-methods-in-python-with-keras/ (accessed Dec. 20, 2021).

Caelen, O. (2017). A Bayesian interpretation of the confusion matrix. *Annals of Mathematics and Artificial Intelligence*, *81*(3–4), 429–450. https://doi.org/10.1007/s10472-017-9564-8

Carneiro, T., Da Nobrega, R. V. M., Nepomuceno, T., Bian, G. Bin, De Albuquerque, V. H. C., & Filho, P. P. R. (2018). Performance Analysis of Google Colaboratory as a Tool for Accelerating Deep Learning Applications. *IEEE Access*, *6*, 61677–61685. https://doi.org/10.1109/ACCESS.2018.2874767

Chollet, F. (2015). *About Keras*. Keras Special Interest Group. https://keras.io/about/

Chollet, F. (2016). Xception: Deep Learning with Depthwise Separable Convolutions. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, *2017-Janua*, 1800–1807. https://doi.org/10.1109/CVPR.2017.195

Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017*, *2017-Janua*, 1800–1807. https://doi.org/10.1109/CVPR.2017.195

Cordeiro de Amorim, R. (2019). Unsupervised feature selection for large data sets. *Pattern Recognition Letters*, *128*(August), 183–189. https://doi.org/10.1016/j.patrec.2019.08.017

Cozzolino, D., Poggi, G., & Verdoliva, L. (2017). Recasting residual-based local descriptors as convolutional neural networks: An application to image forgery detection. *IH and MMSec 2017 - Proceedings of the 2017 ACM Workshop on Information Hiding and Multimedia Security*, *Section 5*, 159–164. https://doi.org/10.1145/3082031.3083247

Dang, H., Liu, F., Stehouwer, J., Liu, X., & Jain, A. K. (2020). On the Detection of Digital Face Manipulation. *Proceedings of the IEEE Computer Society*

Conference on Computer Vision and Pattern Recognition, 5780–5789. https://doi.org/10.1109/CVPR42600.2020.00582

Dave, V. (2020). Spoof Detection Using Local Binary Pattern In Face. *Jurnal Ilmu Komputer*, *13*(1), 39. https://doi.org/10.24843/jik.2020.v13.i01.p05

De, T., Pereira, F., Komulainen, J., Anjos, A., De Martino, J. M., Hadid, A., Pietikäinen, M., & Marcel, S. (2014). Face liveness detection using dynamic texture. *EURASIP Journal on Image and Video Processing*, *2*, 1–15. http://jivp.eurasipjournals.com/content/2014/1/2

Dodge, A., & Johnstone, E. (2018). *Using Fake Video Technology To Perpetuate Intimate Partner Abuse*. 1–9.

Du, J. (2020). High-Precision Portrait Classification Based on MTCNN and Its Application on Similarity Judgement. *Journal of Physics: Conference Series*, *1518*(1). https://doi.org/10.1088/1742-6596/1518/1/012066

Francois Chollet. (2016). Building powerful image classification models using very little data. In *The Keras Blog*. https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html

Ghosh, A., Sufian, A., Sultana, F., Chakrabarti, A., & De, D. (2020). *Fundamental Concepts of Convolutional Neural Network* (Issue June). https://doi.org/10.1007/978-3-030-32644-9

Goodfellow, I. (2016). *01 - Slides - Introduction*.

Google- developers. (2020). *Classification: ROC Curve and AUC | Machine Learning Crash Course*. Developers.Google.Com. https://developers.google.com/machine-learning/crash-course/classification/roc-and-auc

Google. (2020). *Descending into ML: Training and Loss*. Machine Learning Crash Course. https://developers.google.com/machine-learning/crash-course/descending-into-ml/training-and-loss

Grandperrin, J. (2021). *How to use confidence scores in machine learning models | by Jonathan Grandperrin | Towards Data Science*. https://towardsdatascience.com/how-to-use-confidence-scores-in-machine-learning-models-abe9773306fa

Habibzadeh, F., Habibzadeh, P., & Yadollahie, M. (2016). On determining the most appropriate test cut-off value: The case of tests with continuous results. *Biochemia Medica*, *26*(3), 297–307. https://doi.org/10.11613/BM.2016.034

Hasnain, M., Pasha, M. F., Ghani, I., Imran, M., Alzahrani, M. Y., & Budiarto, R. (2020). Evaluating Trust Prediction and Confusion Matrix Measures for Web Services Ranking. *IEEE Access*, *8*, 90847–90861. https://doi.org/10.1109/ACCESS.2020.2994222

John B. Rollins. (2015). Foundational Methodology for Data Science. *IBM Analytics*, 1–4.

Kaur, S., & Kaur, P. (2015). Review and Analysis of Various Image Enhancement Techniques. *International Journal of Computer Applications Technology and Research*, *4*(5), 414–418. https://doi.org/10.7753/ijcatr0405.1016

Khalil, S. S., Youssef, S. M., & Saleh, S. N. (2021). Article icaps-dfake: An integrated capsule-based model for deepfake image and video detection. *Future Internet*, *13*(4). https://doi.org/10.3390/fi13040093

Kim, K. G. (2016). Book Review: Deep Learning. *Healthcare Informatics Research*, *22*(4), 351. https://doi.org/10.4258/hir.2016.22.4.351

Krishna, M. M., Neelima, M., Harshali, M., & Rao, M. V. G. (2018). Image classification using Deep learning. *International Journal of Engineering and Technology(UAE)*, *7*(August), 614–617. https://doi.org/10.14419/ijet.v7i2.7.10892

Kumar, A., Bhavsar, A., & Verma, R. (2020). Detecting Deepfakes with Metric Learning. *2020 8th International Workshop on Biometrics and Forensics*,

IWBF 2020 - Proceedings. https://doi.org/10.1109/IWBF49977.2020.9107962

Lecun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444. https://doi.org/10.1038/nature14539

Li, Y., & Lyu, S. (2018). *Exposing DeepFake Videos By Detecting Face Warping Artifacts*. http://arxiv.org/abs/1811.00656

Li, Y., Yang, X., Sun, P., Qi, H., & Lyu, S. (2019). *Celeb-DF: A New Dataset for DeepFake Forensics*. http://arxiv.org/abs/1909.12962

Li, Y., Yang, X., Sun, P., Qi, H., & Lyu, S. (2020). Celeb-DF: A Large-Scale Challenging Dataset for DeepFake Forensics. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 3204–3213. https://doi.org/10.1109/CVPR42600.2020.00327

Lionnie, R., Bahaweres, R. B., Attamimi, S., & Alaydrus, M. (2017). A study on pre-processing methods for copy-move forgery detection based on SIFT. *IEEE Region 10 Annual International Conference, Proceedings/TENCON*, *2017-Decem*, 1142–1147. https://doi.org/10.1109/TENCON.2017.8228029

Madiraju, N. (2020). *Splitting a dataset. Here I explain how to split your data…*. Towards Data Science. https://towardsdatascience.com/splitting-a-dataset-e328dab2760a

Maggiori, E., Member, S., Tarabalka, Y., Charpiat, G., & Alliez, P. (2016). *Convolutional Neural Networks for Large-Scale Remote-Sensing Image Classification*. 1–13.

Maras, M. H., & Alexandrou, A. (2019). Determining authenticity of video evidence in the age of artificial intelligence and in the wake of Deepfake videos. *International Journal of Evidence and Proof*, *23*(3), 255–262. https://doi.org/10.1177/1365712718807226

Montserrat, D. M., Hao, H., Yarlagadda, S. K., Baireddy, S., Horváth, R. S. J.,

Bartusiak, E., Yang, J., Güera, D., Zhu, F., & Delp, E. J. (2020). Deepfakes detection with automatic face weighting. *ArXiv*.

Nguyen, T. T., Nguyen, C. M., Nguyen, D. T., Nguyen, D. T., & Nahavandi, S. (2019). *Deep Learning for Deepfakes Creation and Detection*. 1–16. http://arxiv.org/abs/1909.11573

P, I. W. S. E., Wijaya, A. Y., & Soelaiman, R. (2016). *Klasifikasi Citra Menggunakan Convolutional Neural Network (Cnn) pada Caltech 101*. *5*(1).

Padmaja, V. K., Srinivas, S. J., Lokesh, S., & Engineering, C. (2020). *Image enhancement using gaussian filters*. *May*, 3268–3270.

Patrini, G., Cavalli, F., & Jder, H. (2018). Report 2018: The State of Deepfakes: Reality Under. *Deeptrace Labs*. https://s3.eu-west-2.amazonaws.com/rep2018/2018-the-state-of-deepfakes.pdf

Perez, J. A., Deligianni, F., Ravi, D., & Yang, G.-Z. (2018). *Artificial Intelligence and Robotics*. *51*(1), 62–67. http://arxiv.org/abs/1803.10813

Prakasa, E. (2016). *Ekstraksi Ciri Tekstur dengan Menggunakan Local Binary Pattern Texture Feature Extraction by Using Local Binary Pattern*. *9*(2), 45–48.

Putra, J. W. G. (2019). *Pengenalan Konsep Pembelajaran Mesin dan Deep Learning*. *4*, 1–235.

Rahmouni, N., Nozick, V., Yamagishi, J., & Echizen, I. (2017). Distinguishing computer graphics from natural images using convolution neural networks. *2017 IEEE Workshop on Information Forensics and Security, WIFS 2017*, *2018-Janua*, 1–6. https://doi.org/10.1109/WIFS.2017.8267647

Ravi, P., & Ashokkumar, A. (2017). Analysis of Various Image Processing Techniques. *International Journal of Advanced Networking & Applications*, *08*(05), 86–89.

Rezende, E., Ruppert, G., Carvalho, T., Ramos, F., & De Geus, P. (2017). Malicious software classification using transfer learning of ResNet-50 deep neural network. *Proceedings - 16th IEEE International Conference on Machine Learning and Applications, ICMLA 2017, 2017-Decem*(December), 1011–1014. https://doi.org/10.1109/ICMLA.2017.00-19

Rodriguez, A. M., Koopman, M., Macarulla Rodriguez, A., & Geradts, Z. (2018). Detection of Deepfake Video Manipulation View project Computational Forensics View project Detection of Deepfake Video Manipulation. *In The 20th Irish Machine Vision and Image Processing Conference (IMVIP), December*, 133–136. http://www.cost.eu/COST_Actions/ca/CA17124

Rosebrock, A. (2019). *Video classification with Keras and Deep Learning*. Pyimagesearch. https://www.pyimagesearch.com/2019/07/15/video-classification-with-keras-and-deep-learning/

Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Niessner, M. (2019). FaceForensics++: Learning to detect manipulated facial images. *Proceedings of the IEEE International Conference on Computer Vision*, *2019-Octob*, 1–11. https://doi.org/10.1109/ICCV.2019.00009

Siddiqui, T. A., Bharadwaj, S., Dhamecha, T. I., Agarwal, A., Vatsa, M., Singh, R., & Ratha, N. (2016). Face anti-spoofing with multifeature videolet aggregation. *Proceedings - International Conference on Pattern Recognition*, *0*, 1035–1040. https://doi.org/10.1109/ICPR.2016.7899772

Spivak, R. (2019). "Deepfakes": the Newest Way To Commit One of the Oldest Crimes. *Georgetown Law Technology Review*, *3.2*(December 2005), 339–400. https://www.lawfareblog.com/deep-fakes-looming-crisis-national-security-democracy-

Sultana, F., Sufian, A., & Dutta, P. (2018). *Advancements in Image Classification using Convolutional Neural Network*.

TensorFlow Core. (2021). *Image classification | TensorFlow Core*. TensorFlow

Core. https://tensorflow.google.cn/tutorials/images/classification

Tutorialspoint. (2019). Tensorflow tutorialspoint. *Tutorials Point (I) Pvt. Ltd.*, 1–13.

Vakili, M., Ghamsari, M., & Rezaei, M. (2020). *Performance Analysis and Comparison of Machine and Deep Learning Algorithms for IoT Data Classification*. Machine Learning Crash Course. http://arxiv.org/abs/2001.09636

Vyas, A., Yu, S., & Paik, J. (2018). Fundamentals of digital image processing. In *Signals and Communication Technology*. https://doi.org/10.1007/978-981-10-7272-7_1

Wahyu, T. (2020). *Digital Transformation : Google Colaboratory sebagai Aplikas...* https://dqlab.id/digital-transformation-mengenal-tools-open-sournce

Wei, X., Yu, X., Liu, B., & Zhi, L. (2019). Convolutional neural networks and local binary patterns for hyperspectral image classification. *European Journal of Remote Sensing*, *52*(1), 448–462. https://doi.org/10.1080/22797254.2019.1634980

Wisskirchen, G., Thibault, B., Bormann, B. U., Muntz, A., Niehaus, G., Soler, G. J., & Von Brauchitsch, B. (2017). Artificial Intelligence and Robotics and Their Impact on the Workplace. *IBA Global Employment Institute*, *April*, 120.

Yang, S., & Berdine, G. (2017). The receiver operating characteristic (ROC) curve. *The Southwest Respiratory and Critical Care Chronicles*, *5*(19), 34. https://doi.org/10.12746/swrccc.v5i19.391

Zhang, H., Qu, Z., Yuan, L., & Li, G. (2017). A face recognition method based on LBP feature for CNN. *Proceedings of 2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference, IAEAC 2017*, 544–547. https://doi.org/10.1109/IAEAC.2017.8054074

Zhang, K., Zhang, Z., Li, Z., Member, S., Qiao, Y., & Member, S. (2016). (MTCNN) Multi-task Cascaded Convolutional Networks. *IEEE Signal Processing Letters*, *23*(10), 1499–1503.

LAMPIRAN

Lampiran 1 : Lampiran Source Code Fungsi Klasifikasi Video Deepfake

```
def klass_vid (vid):
      # create the detector, using default weights
      detector = MTCNN()

      x=1
      k=1
      _video = cv.VideoCapture(vid)
      print('video =', vid)
      print("Processing.....")

      start_time = time.time()
      writer = None
      filenamepre = None
      name = os.path.basename(vid)
      name = name.replace(name[-4:],"")
      m=0
      fps = _video.get(cv.CAP_PROP_FPS)
      max = 0
      maxlabel = 0.0
      Q = deque(maxlen=3)
      print("fps =",fps)

      while (_video):

          success,image = _video.read()
          if (success == False):
                if k==1 :
                  print("Gagal")
                  break
                break

          count = 1
          # detect faces in the image
          faces = detector.detect_faces(image)

          for i in faces :
              x1, y1, w, h = i['box']
              x2, y2 = x1 + w, y1 + h
              if (count > 1) :
                    break
```

```
            try:
              framename = os.path.join(frames_path,
                                       "frame_" + str(k) + ".jpg")
              cv.imwrite(framename,image)
              shape = [(x1, y1), (x2, y2)]
              # create  rectangleimage
              img_box = Image.open(framename)
              img_draw = ImageDraw.Draw(img_box)
              img_draw.rectangle(shape, outline='red', width = 3)
              img_box = img_box.save(framename)

              image_w = cv.resize(image[y1:y2, x1:x2],
                          (224,224), interpolation = cv.INTER_AREA)
              filename = os.path.join(wajah_path, "wajah_" + str(k)
                                                        + ".jpg")
              cv.imwrite(filename,image_w)
              count+=1

              image_w = cv.imread(filename)
              image_pre = process(image_w)
              filenamepre = os.path.join(
                     wajah_path_pre, "wajah_LBP_" + str(k) + ".jpg")
              plt.imsave(filenamepre,image_pre,cmap = 'gray')
              k +=1
            except cv.error as e:
              print('Invalid frame!')

          imgbox_ = cv.imread(framename)
          frame_ = imgbox_.copy()
          if filenamepre != None :
              fr = os.path.basename(framename)
              fr = fr[-5:-4]
              pr = os.path.basename(filenamepre)
              pr = pr[-5:-4]
              if fr == pr :
                data_path = filenamepre
                prediction = load_model.predict([prepare(data_path)])
                score = (prediction[0])
                max = max + (100 * np.max(score))
                m +=1
                maxlabel = maxlabel + score
                Q.append(score)
            # perform prediction averaging over the current history of
            # previous predictions
                results = np.array(Q).mean(axis=0)
```

```
                i = np.argmax(results)
                lab_ = label[i]
                pred = ("{}" .format(lab_))
                cv.putText(frame_, pred, (35, 50),
                    cv.FONT_HERSHEY_SIMPLEX, 1.25, (0,0,255), 5)
            vid_name = os.path.join(vid_path, "klsf_"+name+".mp4")
            h,w,_= image.shape
            size = (w, h)
            # check if the video writer is None
            if writer is None:
                # initialize video writer
                fourcc = cv.VideoWriter_fourcc(*"MJPG")
                writer = cv.VideoWriter(vid_name,fourcc,fps, size, True)
            # write the output frame to disk
            writer.write(frame_)
        writer.release()
        _video.release()
        total_frame = k-1
        print(" ")
        print("total frame :",total_frame)
        print("Waktu Proses :", time.time() - start_time, " detik")
        vide_name = os.path.join(vid_path, "klsf_{}_{:.2f}%_"
               .format(label[np.argmax(maxlabel/m)], max/m)+name+".mp4")
        os.rename(vid_name,vide_name)
        print("Disimpan :", vide_name)
        print('Proses Menyimpan.....')
        for i in tqdm(range(0,140)) :
            time.sleep(1)
            if i == 139 :
              print("\nselesai")
```